Applied Mathematics

A Hot-Started Nullspace Active-Set Method for Regularized Column Generation

with Equality and Inequality Linking Constraints

Author: *Kyrian Bäumer*

October 10, 2026

Abstract

Column generation solves linear programs with a very large number of variables by working on a restricted master problem that is enlarged one column at a time. Its classical form is known for unstable dual iterates and slow convergence. A quadratic proximal term in the dual space removes most of this instability, but it turns every master problem into a convex quadratic program, which is considerably more expensive to solve than a linear program.

This work develops a **hot-started nullspace active-set method** for these quadratic master problems, for multi-block problems whose linking constraints may be equalities, "$\geq$" inequalities or "$\leq$" inequalities. Surplus variables bring all linking rows into one equality form. The convexity constraints of the blocks then admit a reordering of the variables into five sets, which yields an explicit and sparse nullspace basis. The reduced KKT system decouples: the surplus part of the search direction is given by an explicit formula, and only an *effective reduced Hessian* on the column weights has to be factorized. This matrix is the reduced Hessian of the equality rows together with the inequality rows whose surplus is fixed at zero. Every change of the working set becomes a low-rank modification of a fixed-size $LDL^\top$ factorization, which is updated in place with the sparse direct solver CHOLMOD and carried over from one column generation iteration to the next. The Lagrange multipliers of the method coincide with the reduced costs of column generation and with the dual sign conditions of the inequality rows. Singular reduced Hessians are resolved by an exact zero-curvature step, and periodic refactorization controls the structural zeros that accumulate in the factor.

On generated airline crew scheduling instances the method reproduces the optimum of the full linear program, and it solves the regularized master problems two to three orders of magnitude faster than a general-purpose quadratic programming solver. The complete column generation run is also much faster than classical column generation with a linear master problem, and on the larger instances faster than solving the full linear program directly.

Contents

Abstract iii

1 Introduction 1

1.1 The Computational Problem . . . 2
1.2 Contributions . . . 2
1.3 Outline . . . 3

2 Problem Formulation 5

2.1 Multi-Block Linear Programs with Mixed Linking Constraints . . . 6
2.2 Dantzig–Wolfe Reformulation . . . 7
2.3 Column Generation . . . 9
2.4 Surplus Variables and the Unified Formulation . . . 10
2.5 Summary of Notation . . . 11

3 Regularization of the Master Problem 13

3.1 Proximal Stabilization of the Dual . . . 14
3.2 The Explicit Regularized Master Problem . . . 14
3.3 The Eliminated Regularized Master Problem . . . 15
3.3.1 The dual vector as a function of the primal iterate . . . 15
3.4 Regularized Column Generation . . . 16
3.4.1 Center updates . . . 17
3.4.2 Exact termination . . . 17
3.4.3 The Lagrangian bound with inequality rows . . . 18
3.4.4 The algorithm . . . 18

4 An Active-Set Method for the Regularized Master Problem 19

4.1 The Master Problem in Simplified Notation . . . 20
4.2 Optimality Conditions . . . 20
4.3 The Primal Active-Set Method . . . 21
4.4 The Constraint Matrix and the Reordering Theorem . . . 22

5 The Nullspace Method 25

5.1 The Nullspace Basis . . . 26

5.2 The Reduced KKT System . . . 27
5.2.1 The reduced system . . . 27
5.2.2 Block structure . . . 27
5.2.3 Decoupling and the effective reduced Hessian . . . 28
5.2.4 The search direction . . . 29
5.2.5 Implicit evaluation . . . 30
5.3 The Lagrange Multipliers . . . 30
5.4 Summary of One Iteration . . . 31

6 Efficient Updates of the KKT System **33**
6.1 Fixed-Length Representation . . . 34
6.2 Expanding the Working Set . . . 35
6.3 Reducing the Working Set . . . 36
6.4 Updating the $LDL^\top$ Factorization . . . 37
6.4.1 Row additions and deletions . . . 37
6.4.2 Rank-one modifications . . . 37
6.4.3 Sparse solves . . . 38
6.4.4 Assembly of a new row . . . 38
6.5 Structural Zeros and Periodic Refactorization . . . 38
6.6 Hot Start Between Column Generation Iterations . . . 39
6.6.1 Center-only iterations . . . 39
6.7 The Complete Method . . . 40

7 Singular Reduced Hessians **43**
7.1 Where Singularity Can Arise . . . 44
7.2 Detection . . . 44
7.3 Zero-Curvature Directions . . . 44
7.3.1 Released column . . . 45
7.3.2 Released surplus . . . 45
7.4 The Zero-Curvature Step Removes the Singularity . . . 46

8 Computational Results **49**
8.1 Test Problem: Airline Crew Scheduling . . . 50
8.1.1 The model . . . 50
8.1.2 Instance generator . . . 50
8.2 Implementation and Setup . . . 51
8.3 Correctness . . . 51
8.4 Convergence and the Two Phases . . . 54
8.5 Computational Time . . . 56
8.6 Inequality Rows . . . 59

8.7 Fill-In and Refactorization 59
8.8 Discussion 61

9 Conclusion **63**
9.1 Summary 63
9.2 Outlook 64

Bibliography **65**

Chapter 1

Introduction

Many large-scale planning problems, such as crew scheduling, vehicle routing and cutting stock, lead to linear programs with a block structure: a number of independent subproblems, one per crew member, vehicle or machine, are coupled by a comparatively small set of **linking constraints**. In crew scheduling, for instance, every pilot follows a schedule of their own, while the linking constraints require that each flight is covered by the pilots together.

Column generation is the standard method for such problems. The problem is first rewritten in terms of complete solutions of the subproblems (a schedule, a route, a cutting pattern), so that every variable, or *column*, represents one such solution. There are far too many columns to write down, but only a few of them are needed in an optimal solution. Column generation therefore works with a **restricted master problem** over a small set of columns and asks the subproblems, in the **pricing problem**, for a new column that can improve the current solution.

In its classical form the restricted master problem is a linear program. Its dual solutions, which drive the pricing problem, are known to jump from one extreme point to another, which makes the method unstable and slow in its final phase [14]. A well-established remedy is **proximal stabilization** [8, 2]: the dual problem is augmented by a quadratic term that keeps the dual iterate close to a *stability center*. The price of this regularization is that every restricted master problem becomes a **convex quadratic program**.

This work is about solving this sequence of quadratic programs efficiently, for linking constraints that may be equalities as well as inequalities.

1.1 The Computational Problem

Two consecutive restricted master problems in column generation are almost identical. The new problem contains one more column, and with stabilization the stability center may have moved. A general-purpose quadratic programming solver does not exploit this: it rebuilds and refactorizes its internal linear algebra for every master problem, and its cost per master problem grows with the number of generated columns.

Active-set methods are well suited for sequences of closely related quadratic programs. They move between faces of the feasible region by adding or removing one constraint at a time, and the linear system they solve at each step changes only by a low-rank term. In the **nullspace** variant, this system is the reduced Hessian on the nullspace of the active constraints. Its size equals the number of degrees of freedom of the current face, not the number of variables.

The question addressed in this work is how far this structure can be exploited for the regularized master problem of column generation, in particular:

(i) What is the structure of the active constraints of the regularized master problem, and does it allow an explicit nullspace basis?

(ii) How small is the linear system that has to be factorized, and how does it change when the working set changes?

(iii) Can the factorization be carried over from one column generation iteration to the next, so that a new column or a new stability center costs (almost) nothing?

(iv) What changes when the linking constraints are inequalities, which add sign conditions on the dual variables and destroy part of the structure?

(v) How does the resulting method compare in practice with general-purpose solvers and with classical column generation?

1.2 Contributions

The main results of this work are the following.

- **A unified formulation for mixed linking constraints.** Equality rows and inequality rows of both directions are written as equalities with surplus variables. The regularized master problem then has a single form for all cases (Chapters 2 and 3).
- **A five-set column ordering and an explicit nullspace basis.** The convexity constraints of the blocks, together with the surplus variables, admit an ordering of the variables into representatives, working-set columns, nullspace columns, active surpluses and free surpluses. The nullspace basis consists of vectors $e_j - e_{\mathrm{rep}(j)}$ and unit vectors, and it is never stored (Chapters 4 and 5).
- **Decoupling and the effective reduced Hessian.** The reduced KKT system splits into an explicit formula for the surplus directions and a system with the *effective reduced Hessian* $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$. Here A_{eff} holds the equality rows together with the inequality rows whose surplus is fixed at zero. Its dimension never exceeds the number of linking constraints (Chapter 5).

- **Multipliers are reduced costs.** The Lagrange multipliers of the active-set method are exactly the reduced costs of column generation and the dual sign conditions of the inequality rows. The optimality test of the master problem and the pricing test are therefore the same test, applied inside and outside the restricted master problem (Chapter 5).
- **Updates and hot start.** Every change of the working set is a row addition, a row deletion or a rank-one modification of a fixed-size $LDL^\top$ factorization. A new column or a new stability center leaves the factorization unchanged (Chapter 6).
- **Exact treatment of singular reduced Hessians.** A dependent nullspace direction is resolved by a zero-curvature step that keeps every working-set variable exactly at its bound (Chapter 7).
- **A computational study** on generated crew scheduling instances with equality and inequality coverage rows: correctness against the full linear program, timing, the two phases of convergence, and the effect of refactorization on the fill of the factor (Chapter 8).

1.3 Outline

Chapter 2 introduces the multi-block linear program with mixed linking constraints, its Dantzig–Wolfe reformulation, column generation and the surplus formulation. Chapter 3 adds the proximal regularization, eliminates the auxiliary variable and derives the relation between the gradient of the quadratic master problem and the reduced costs. Chapter 4 sets up the active-set method for the regularized master problem and establishes the five-set reordering of the constraint matrix. Chapter 5 constructs the nullspace basis, solves the reduced KKT system with the effective reduced Hessian and recovers the Lagrange multipliers. Chapter 6 describes the updates of the factorization when the working set changes, the fixed-length representation, periodic refactorization and the hot start between column generation iterations. Chapter 7 treats singular reduced Hessians. Chapter 8 presents the computational results, and Chapter 9 concludes.

Throughout, the reader is assumed to be familiar with linear programming duality, the basic idea of column generation and the basic idea of a primal active-set method for quadratic programming, as found in standard texts such as [15, 7]. Everything beyond this is derived in the text.

Chapter 2

Problem Formulation

This chapter introduces the class of problems considered throughout this work: linear programs with a multi-block structure, whose blocks are coupled by linking constraints that may be equalities or inequalities.

We first state the problem in its original (compact) form and then apply the **Dantzig–Wolfe reformulation**, which replaces the variables of each block by convex combinations of the extreme points of the block. This leads to the **master problem**, a linear program with a very large number of columns, which is solved by **column generation**. We define the restricted master problem, the reduced cost, the pricing problem and the Lagrangian bound, and collect them in the classical column generation algorithm.

Finally, we introduce **surplus variables** for the inequality rows. They bring every linking constraint into equality form and lead to a single, *unified* formulation of the master problem in which equality and inequality rows are treated alike. This formulation is the starting point for everything that follows. It also reveals the one structural property that the inequality rows destroy: the surplus variables belong to no block.

2.1 Multi-Block Linear Programs with Mixed Linking Constraints

We consider a linear program whose variables are partitioned into K blocks $x_1, \dots, x_K$, with $x_k \in \mathbb{R}^{n_k}$. Each block has its own constraints, and the blocks interact only through a set of linking constraints.

Definition 2.1.1 (Multi-block linear program)**.** A **multi-block linear program with mixed linking constraints** is a problem of the form

$$\min_{x_1,\dots,x_K} \quad \sum_{k=1}^{K} c_k^\top x_k \tag{2.1a}$$

$$\text{s.t.} \quad \sum_{k=1}^{K} A_{E,k}\, x_k = b_E, \tag{2.1b}$$

$$\sum_{k=1}^{K} A_{I,k}\, x_k \geq b_I, \tag{2.1c}$$

$$x_k \in P_k, \qquad k = 1, \dots, K, \tag{2.1d}$$

where $A_{E,k} \in \mathbb{R}^{m_E \times n_k}$ and $A_{I,k} \in \mathbb{R}^{m_I \times n_k}$ are the coefficients of block k in the m_E **equality linking rows** and the m_I **inequality linking rows**, and

$$P_k = \{\, x_k \in \mathbb{R}^{n_k} \;:\; F_k x_k = d_k,\; l_k \leq x_k \leq u_k \,\}$$

is the **block polytope** of block k. We write $m = m_E + m_I$ for the total number of linking rows. Constraints (2.1b)–(2.1c) are the **linking constraints**, and constraints (2.1d) are the **block constraints**.

Assumption 2.1.2. Every block polytope P_k is nonempty and bounded, and problem (2.1) is feasible.

Under Assumption 2.1.2 the feasible region of (2.1) is a nonempty polytope, so the problem has an optimal solution. Its optimal value is denoted by $z^\star$.

The constraint matrix of (2.1) has the **block-angular** structure shown in Fig. 2.1. The block constraints form a block-diagonal matrix, and only the linking rows couple different blocks. If the linking rows were absent, the problem would decompose into K independent problems over $P_1, \dots, P_K$. Column generation exploits exactly this observation.

Remark 2.1.3 (Inequalities of the other direction)**.** A linking row of the form $\sum_k a_{i,k}^\top x_k \leq b_i$ is the row $\sum_k (-a_{i,k})^\top x_k \geq -b_i$, so the form (2.1c) covers both directions. For the presentation we therefore only consider "$\geq$" rows. In Section 2.4 we show that a "$\leq$" row can equivalently be kept as it is, with a surplus variable of the opposite sign. This is how such rows are handled in the implementation.

Example 2.1.4 (Crew scheduling)**.** In the crew scheduling problem studied in Chapter 8, block k is the schedule of pilot k: a path from a source to a sink in a time–space network, with F_k the node–edge incidence matrix of the network. The linking rows are the *coverage rows*: row i counts how often flight i is operated, summed over all pilots. An equality row "$= 1$" requires that the flight is operated exactly once, and an inequality row "≥ 1" requires it at least once, which allows a pilot to travel on a flight as a passenger.

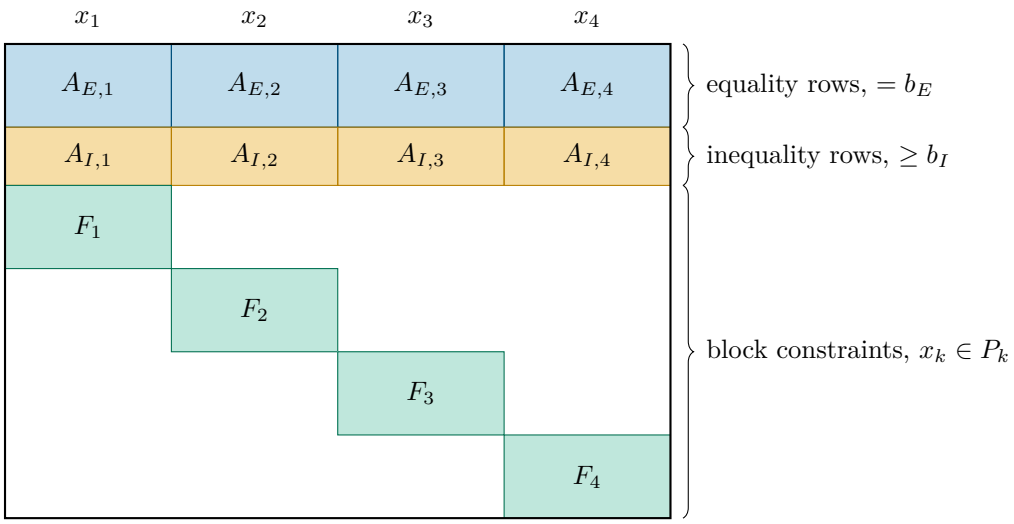


Figure 2.1: Block-angular structure of the multi-block linear program (2.1) for $K = 4$ blocks. The linking rows (top) couple all blocks; the block constraints (bottom) are block-diagonal.

2.2 Dantzig–Wolfe Reformulation

Since P_k is a nonempty polytope, it is the convex hull of its finitely many extreme points (Minkowski's theorem). Let $\mathcal{J}_k$ be an index set of the extreme points of P_k, and let $x^{(j)}$, $j \in \mathcal{J}_k$, denote these extreme points. Every $x_k \in P_k$ can be written as

$$x_k = \sum_{j \in \mathcal{J}_k} \lambda_j \, x^{(j)}, \qquad \sum_{j \in \mathcal{J}_k} \lambda_j = 1, \qquad \lambda_j \geq 0. \tag{2.2}$$

Substituting (2.2) into (2.1) replaces the variables x_k by the weights λ_j. Each extreme point turns into one column of a new problem, which motivates the following definitions.

Definition 2.2.1 (Columns)**.** Let $\mathcal{J} = \mathcal{J}_1 \cup \cdots \cup \mathcal{J}_K$ (disjoint union). For $j \in \mathcal{J}_k$ we define

- the **block** of column j: $\psi(j) = k$;
- the **cost** of column j: $c_j = c_k^\top x^{(j)}$;
- the **linking coefficients** of column j: $a_j^E = A_{E,k}\, x^{(j)} \in \mathbb{R}^{m_E}$, $a_j^I = A_{I,k}\, x^{(j)} \in \mathbb{R}^{m_I}$, and $a_j = [\, a_j^E;\, a_j^I \,] \in \mathbb{R}^m$.

We collect them in the cost vector $c = (c_j)_{j \in \mathcal{J}}$ and the matrices

$$A_E = [\, a_j^E \,]_{j \in \mathcal{J}} \in \mathbb{R}^{m_E \times |\mathcal{J}|}, \qquad A_I = [\, a_j^I \,]_{j \in \mathcal{J}} \in \mathbb{R}^{m_I \times |\mathcal{J}|}, \qquad A = \begin{bmatrix} A_E \\ A_I \end{bmatrix} \in \mathbb{R}^{m \times |\mathcal{J}|}.$$

Definition 2.2.2 (Convexity matrix)**.** The **convexity matrix** $C \in \{0,1\}^{K \times |\mathcal{J}|}$ has the entries $C_{kj} = 1$ if $\psi(j) = k$ and $C_{kj} = 0$ otherwise. The constraint $C\lambda = \mathbf{1}$ states that the weights of the columns of every block sum to one.

Definition 2.2.3 (Master problem)**.** The **master problem** is the linear program

$$\min_{\lambda} \quad c^\top \lambda \tag{2.3a}$$

$$\text{s.t.} \quad A_E\, \lambda = b_E, \qquad (\pi_E) \tag{2.3b}$$

$$A_I\, \lambda \geq b_I, \qquad (\pi_I) \tag{2.3c}$$

$$C\lambda = \mathbf{1}, \qquad (\mu) \tag{2.3d}$$

$$\lambda \geq 0, \tag{2.3e}$$

where the symbols in parentheses denote the dual variables of the constraints.

Theorem 2.2.4 (Dantzig–Wolfe [4])**.** Under Assumption 2.1.2, problem (2.3) has the optimal value $z^\star$. If λ is optimal for (2.3), then $x_k = \sum_{j \in \mathcal{J}_k} \lambda_j \, x^{(j)}$, $k = 1, \dots, K$, is optimal for (2.1).

Proof. By (2.2), $x_k \in P_k$ if and only if x_k is a convex combination of the $x^{(j)}$, $j \in \mathcal{J}_k$. Under this substitution the objective and the linking rows are linear in λ, with exactly the coefficients of Definition 2.2.1. The feasible points of (2.1) are therefore the images of the feasible points of (2.3), with equal objective values. □

The convexity matrix has a very particular structure, which will be essential for the nullspace method of Chapter 5.

Definition 2.2.5 (Column generation property)**.** A matrix $C \in \mathbb{R}^{K \times n}$ has the **column generation property** if

(i) every entry of C is 0 or 1;

(ii) every column of C contains exactly one entry equal to 1;

(iii) every row of C contains at least one entry equal to 1.

Lemma 2.2.6. The convexity matrix C of Definition 2.2.2 has the column generation property, and so does every submatrix of C formed by a subset of its columns that contains at least one column of every block.

Proof. Property (i) holds by definition. Every column belongs to exactly one block, which gives property (ii) for every subset of columns. Property (iii) holds for a subset of columns if and only if every block is represented in it. □

Lemma 2.2.7. If $C \in \mathbb{R}^{K \times n}$ has the column generation property, then $\operatorname{rank}(C) = K$.

Proof. By property (iii), for every row k there is a column j_k with $C_{k j_k} = 1$. By property (ii), this column has no other nonzero entry, i.e. it equals the unit vector e_k. The columns $j_1, \dots, j_K$ therefore form the identity matrix I_K. □

The dual of the master problem

The dual of (2.3) has one constraint per column and the sign condition of the inequality rows:

$$\max_{\pi_E, \pi_I, \mu} \quad b_E^\top \pi_E + b_I^\top \pi_I + \mathbf{1}^\top \mu \tag{2.4a}$$

$$\text{s.t.} \quad c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0, \qquad j \in \mathcal{J}, \tag{2.4b}$$

$$\pi_I \geq 0, \tag{2.4c}$$

where $\pi = [\pi_E; \pi_I] \in \mathbb{R}^m$. Compared with a problem that has only equality linking rows, the inequality rows add the **sign condition** (2.4c). This is the only change in the dual, but much of the work in the following chapters is devoted to it.

2.3 Column Generation

The index set $\mathcal{J}$ is typically far too large to enumerate. In crew scheduling, for example, $\mathcal{J}_k$ contains every feasible schedule of pilot k. Column generation solves (2.3) by working with a small subset of the columns at a time.

Definition 2.3.1 (Restricted master problem)**.** For a subset $\mathcal{J}' \subseteq \mathcal{J}$ that contains at least one column of every block, the **restricted master problem** is problem (2.3) with all columns outside $\mathcal{J}'$ removed (equivalently, with $\lambda_j = 0$ fixed for $j \notin \mathcal{J}'$). We write $n = |\mathcal{J}'|$ for the number of columns in the restricted master problem.

Definition 2.3.2 (Reduced cost)**.** For dual values (π, μ) the **reduced cost** of a column $j \in \mathcal{J}$ is

$$\bar{c}_j(\pi, \mu) \;=\; c_j - a_j^\top \pi - \mu_{\psi(j)}.$$

The reduced cost is the slack of the dual constraint (2.4b). A column with a negative reduced cost violates the dual constraints and can improve the restricted master problem. Finding the column of most negative reduced cost in block k is the **pricing problem**

$$\min_{j \in \mathcal{J}_k} \bar{c}_j(\pi, \mu) \;=\; \min_{x_k \in P_k} \left(c_k - A_k^\top \pi\right)^\top x_k \;-\; \mu_k, \qquad A_k = \begin{bmatrix} A_{E,k} \\ A_{I,k} \end{bmatrix}, \tag{2.5}$$

where the equality holds because a linear function attains its minimum over a polytope at an extreme point. The pricing problem is a problem over a single block. In crew scheduling it is a shortest path problem with edge weights $c_{e,k} - (A_k^\top \pi)_e$.

Proposition 2.3.3 (Optimality test)**.** Let λ' be optimal for the restricted master problem, with optimal duals (π, μ). If $\pi_I \geq 0$ and the pricing problem (2.5) has a nonnegative optimal value for every block, then λ', extended by zeros, is optimal for the master problem (2.3).

Proof. The extended λ' is feasible for (2.3). The pair (π, μ) satisfies the sign condition (2.4c) by assumption and every dual constraint (2.4b) by the pricing result, so it is dual feasible. Complementary slackness holds for the columns in $\mathcal{J}'$ by optimality of the restricted problem, and trivially for the zero weights outside $\mathcal{J}'$. □

For a linear restricted master problem the duals satisfy $\pi_I \geq 0$ automatically, so the pricing test alone decides optimality. With the regularized master problems of the next chapter this will no longer be the case. Column generation also provides a lower bound on $z^\star$ at every iteration in which all blocks are priced.

Proposition 2.3.4 (Lagrangian bound)**.** For every $\pi = [\pi_E; \pi_I]$ with $\pi_I \geq 0$, the **Lagrangian bound**

$$\theta(\pi) \;=\; b^\top \pi + \sum_{k=1}^{K} \min_{x_k \in P_k} \left(c_k - A_k^\top \pi\right)^\top x_k, \qquad b = \begin{bmatrix} b_E \\ b_I \end{bmatrix},$$

satisfies $\theta(\pi) \leq z^\star$.

Proof. Let $x^\star$ be optimal for (2.1). Because $\sum_k A_{E,k} x_k^\star = b_E$, $\sum_k A_{I,k} x_k^\star \geq b_I$ and $\pi_I \geq 0$, we have $\pi^\top (b - \sum_k A_k x_k^\star) \leq 0$. Hence

$$z^\star = \sum_k c_k^\top x_k^\star \;\geq\; \sum_k c_k^\top x_k^\star + \pi^\top \Big(b - \sum_k A_k x_k^\star\Big) = b^\top \pi + \sum_k (c_k - A_k^\top \pi)^\top x_k^\star \;\geq\; \theta(\pi),$$

since $x_k^\star \in P_k$. □

Remark 2.3.5. The sign condition $\pi_I \geq 0$ in Proposition 2.3.4 is essential: for $\pi_i < 0$ in an inequality row, the term $\pi_i \left(b_i - \sum_k a_{i,k}^\top x_k\right)$ can become arbitrarily large. In Section 3.4 the bound is therefore evaluated at the projection of the dual vector onto $\{\pi_I \geq 0\}$.

The resulting method is summarized in Algorithm 2.1. The initial columns are usually obtained by solving the pricing problem once with zero duals, which gives one column per block.

Algorithm 2.1: Classical Column Generation

Input: initial columns $\mathcal{J}'$ with at least one column per block
repeat
  Solve the (linear) restricted master problem over $\mathcal{J}'$; obtain λ' and duals (π, μ)
  for each block $k = 1, \ldots, K$ **do**
    Solve the pricing problem (2.5); let j be the best column
    if $\bar{c}_j(\pi, \mu) < 0$ **then**
      $\mathcal{J}' \leftarrow \mathcal{J}' \cup \{j\}$
    end if
  end for
until no column with negative reduced cost was found
Output: λ', optimal for the master problem by Proposition 2.3.3

Remark 2.3.6 (Instability)**.** The duals of a linear restricted master problem are extreme points of its dual feasible region. When a column is added, the dual solution can jump to a very different extreme point, even if the primal solution hardly changes. As a result, the duals oscillate strongly from one iteration to the next, the pricing problem generates many columns that are never used, and the method converges slowly in its final phase (the *tailing-off effect*). In the dual space, column generation is Kelley's cutting-plane method [12] applied to the Lagrangian dual function, a method known for exactly this behaviour. This is the motivation for the regularization of Chapter 3.

2.4 Surplus Variables and the Unified Formulation

The two kinds of linking rows in (2.3) behave differently: the equality rows fix $A_E\lambda$, while the inequality rows only bound $A_I\lambda$ from below. To treat both in the same way, we introduce for every inequality row i a **surplus variable** $s_i \geq 0$ with

$$(A_I\lambda)_i - s_i = (b_I)_i, \qquad \text{i.e.} \qquad A_I\lambda - s = b_I.$$

The surplus measures how much the row is over-satisfied.

Definition 2.4.1 (Unified formulation)**.** Let $s \in \mathbb{R}^{m_I}$ be the vector of surplus variables. The **unified variable** and the **augmented data** are

$$y = \begin{bmatrix} \lambda \\ s \end{bmatrix}, \qquad \bar{A} = \begin{bmatrix} A_E & 0 \\ A_I & -I \end{bmatrix}, \qquad \bar{C} = \begin{bmatrix} C & 0 \end{bmatrix}, \qquad \bar{c} = \begin{bmatrix} c \\ 0 \end{bmatrix}, \qquad b = \begin{bmatrix} b_E \\ b_I \end{bmatrix}.$$

The **unified master problem** is

$$\min_y \ \bar{c}^\top y \qquad \text{s.t.} \qquad \bar{A}y = b, \quad \bar{C}y = \mathbf{1}, \quad y \geq 0. \tag{2.6}$$

Lemma 2.4.2. Problems (2.3) and (2.6) are equivalent: λ is feasible for (2.3) if and only if $y = [\lambda;\ A_I\lambda - b_I]$ is feasible for (2.6), with the same objective value. Moreover, the dual of (2.6) is the dual (2.4) of the master problem.

Proof. The first statement follows from $s = A_I\lambda - b_I \geq 0$. For the second, the dual of (2.6) has multipliers π for $\bar{A}y = b$ and μ for $\bar{C}y = \mathbf{1}$, and one constraint per column of $[\bar{A}; \bar{C}]$. For the columns of λ these are the constraints (2.4b). The column of surplus s_i has cost 0, coefficient $-e_i$ in the inequality rows and 0 in the convexity rows, so its dual constraint reads $0 - (-e_i)^\top \pi_I - 0 \geq 0$, i.e. $(\pi_I)_i \geq 0$. These are exactly the sign conditions (2.4c). □

Lemma 2.4.2 shows that the surplus columns *encode the sign conditions as ordinary dual constraints*. Problem (2.6) has the same form as a master problem with equality linking rows only, with (A, C, c) replaced by $(\bar{A}, \bar{C}, \bar{c})$. All the following chapters work with this form.

Remark 2.4.3 (The column generation property is lost)**.** The augmented convexity matrix $\bar{C} = [\,C \quad 0\,]$ has m_I zero columns, one per surplus variable, so it violates property (ii) of Definition 2.2.5. The surpluses do not belong to any block. Chapter 4 shows that this causes no difficulty: zero columns are even simpler to handle than unit columns, but they have to be treated as a separate group of variables.

Remark 2.4.4 ("$\leq$" rows and the slack sign)**.** A row $a_i^\top \lambda \leq b_i$ can be written with a *slack* $s_i \geq 0$ as $a_i^\top \lambda + s_i = b_i$. In general, let $\kappa_i \in \{-1, +1\}$ be the coefficient of the slack of inequality row i ($\kappa_i = -1$ for a "$\geq$" row, $\kappa_i = +1$ for a "$\leq$" row) and $K_s = \mathrm{diag}(\kappa)$. The unified formulation then uses

$$\bar{A} = \begin{bmatrix} A_E & 0 \\ A_I & K_s \end{bmatrix}$$

instead of Definition 2.4.1, and the dual constraint of slack i becomes $-\kappa_i(\pi_I)_i \geq 0$, i.e. $(\pi_I)_i \leq 0$ for a "$\leq$" row. Since $\kappa_i^2 = 1$, every result of the following chapters holds for both kinds of rows; only a few formulas carry a factor κ_i, and these are collected in Remark 5.3.3. In the text we write all formulas for "$\geq$" rows ($K_s = -I$).

Remark 2.4.5 (Range rows)**.** A row with two different finite bounds $\ell_i \leq a_i^\top \lambda \leq u_i$ needs a slack with an upper bound. Such rows are not considered in this work.

2.5 Summary of Notation

symbol	meaning
K, k	number of blocks, block index
m_E, m_I, m	number of equality, inequality and all linking rows
$\mathcal{J}$, $\mathcal{J}_k$, $\mathcal{J}'$	all columns, columns of block k, columns of the restricted problem
n	number of columns in the restricted master problem
$\psi(j)$	block of column j
c_j, $a_j = [a_j^E; a_j^I]$	cost and linking coefficients of column j
A_E, A_I, A	linking coefficient matrices of the columns
C	convexity matrix
λ, s, $y = [\lambda; s]$	column weights, surpluses, unified variable
$\bar{A}$, $\bar{C}$, $\bar{c}$	augmented data of the unified formulation
$\pi = [\pi_E; \pi_I]$, μ	duals of the linking rows and of the convexity rows
$\bar{c}_j(\pi, \mu)$	reduced cost of column j
$\theta(\pi)$	Lagrangian bound

Table 2.1: Notation of Chapter 2.

Chapter 3

Regularization of the Master Problem

This chapter replaces the linear restricted master problem by a regularized one. The regularization is formulated in the dual space, where it adds a quadratic penalty on the distance between the dual iterate and a **stability center**. Translated back to the primal space, it yields a convex quadratic program with an auxiliary variable, the **explicit regularized master problem**.

Because all linking rows of the unified formulation are equalities, the auxiliary variable can be eliminated for equality and inequality rows alike. The result, the **eliminated regularized master problem**, is a convex quadratic program whose only constraints are the convexity constraints and the nonnegativity of the variables. This is the problem that the active-set method of the following chapters solves.

Two identities derived in this chapter are used throughout the rest of the work. The first expresses the duals of the linking rows as an explicit function of the primal iterate. The second shows that the gradient of the quadratic objective consists of the reduced costs of the columns and the duals of the inequality rows. The chapter ends with the regularized column generation algorithm, its exact stopping criterion and the accuracy that this criterion guarantees.

3.1 Proximal Stabilization of the Dual

Consider the restricted master problem in the unified form (2.6), over the columns $\mathcal{J}'$ and all m_I surpluses. Its dual is

$$\max_{\pi,\,\mu}\ b^\top\pi + \mathbf{1}^\top\mu \qquad \text{s.t.} \qquad c_j - a_j^\top\pi - \mu_{\psi(j)} \geq 0\ \ (j \in \mathcal{J}'), \qquad \pi_I \geq 0. \tag{3.1}$$

Its optimal dual solutions are extreme points of the feasible region, which causes the instability described in Remark 2.3.6. Proximal stabilization adds a penalty on the distance to a given point $\hat{\pi} \in \mathbb{R}^m$.

Definition 3.1.1 (Stabilized dual problem)**.** Let $\hat{\pi} \in \mathbb{R}^m$ be the **stability center** and $\delta > 0$ the **stabilization parameter**. The **stabilized dual problem** is

$$\max_{\pi,\,\mu}\ b^\top\pi + \mathbf{1}^\top\mu - \frac{\delta}{2}\,\|\pi - \hat{\pi}\|^2 \qquad \text{s.t.} \qquad c_j - a_j^\top\pi - \mu_{\psi(j)} \geq 0\ \ (j \in \mathcal{J}'), \qquad \pi_I \geq 0. \tag{3.2}$$

The objective of (3.2) is strictly concave in π, so the optimal π is unique. A large δ keeps π close to $\hat{\pi}$ (strong stabilization), and $\delta \to 0$ recovers (3.1). Replacing the center $\hat{\pi}$ by the solution π and solving again is one step of the **proximal point method** [16] applied to the restricted dual problem. If the center is only moved when it improves the Lagrangian bound, we obtain a **proximal bundle method** [13, 2]. Both rules are discussed in Section 3.4.

3.2 The Explicit Regularized Master Problem

The primal counterpart of (3.2) has an auxiliary vector $w \in \mathbb{R}^m$, one entry per linking row.

Definition 3.2.1 (Explicit regularized master problem)**.** The **explicit regularized master problem** is

$$\min_{y,\,w}\quad \bar{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2 \tag{3.3a}$$

$$\text{s.t.}\quad \bar{A}y + \delta w = b, \qquad (\pi) \tag{3.3b}$$

$$\bar{C}y = \mathbf{1}, \qquad (\mu) \tag{3.3c}$$

$$y \geq 0. \tag{3.3d}$$

Theorem 3.2.2. The dual of (3.3) is the stabilized dual problem (3.2). Strong duality holds, and at a pair of optimal solutions

$$\pi = \hat{\pi} + w. \tag{3.4}$$

Proof. The Lagrangian of (3.3), with multipliers π, μ and $\nu \geq 0$ for $y \geq 0$, is

$$\mathcal{L} = \bar{c}^\top y + \tfrac{\delta}{2}\|w + \hat{\pi}\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 + \pi^\top(b - \bar{A}y - \delta w) + \mu^\top(\mathbf{1} - \bar{C}y) - \nu^\top y.$$

It is a strictly convex quadratic function of w, minimized where $\delta(w + \hat{\pi}) - \delta\pi = 0$, i.e. at $w = \pi - \hat{\pi}$. Inserting this,

$$\tfrac{\delta}{2}\|\pi\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 - \delta\pi^\top(\pi - \hat{\pi}) = -\tfrac{\delta}{2}\|\pi - \hat{\pi}\|^2.$$

The Lagrangian is linear in y, and its infimum over y is finite only if $\bar{c} - \bar{A}^\top\pi - \bar{C}^\top\mu - \nu = 0$ for some $\nu \geq 0$, i.e. only if $\bar{c} - \bar{A}^\top\pi - \bar{C}^\top\mu \geq 0$. The dual function is therefore $b^\top\pi + \mathbf{1}^\top\mu - \frac{\delta}{2}\|\pi - \hat{\pi}\|^2$, subject to this condition. Written per column, the condition for the columns of λ is (2.4b), and the condition for the surplus columns is $\pi_I \geq 0$ (Lemma 2.4.2). This is (3.2). Problem (3.3) is a feasible convex quadratic program with linear constraints (it is feasible because w is free), so strong duality holds, and the minimizing w satisfies (3.4). □

Remark 3.2.3. Theorem 3.2.2 shows that the sign condition $\pi_I \geq 0$ is part of the stabilized dual problem automatically, through the surplus columns. The stability center $\hat{\pi}$ itself may violate it. The proximal step then moves π back into the region $\pi_I \geq 0$.

Remark 3.2.4. Without surplus variables, an inequality row $(A_I\lambda)_i \geq (b_I)_i$ would enter the explicit problem as $(A_I\lambda)_i + \delta w_i \geq (b_I)_i$, and w_i would no longer be determined by λ. The surpluses make every row of (3.3b) an equality, so that w can be eliminated for all rows in the next section.

3.3 The Eliminated Regularized Master Problem

From (3.3b) we get $w = (b - \bar{A}y)/\delta$ for every feasible point. Substituting this into the objective of (3.3) removes w completely.

Theorem 3.3.1 (Eliminated regularized master problem)**.** Problem (3.3) is equivalent to the convex quadratic program

$$\min_{y} \quad f(y) = \frac{1}{2\delta}\, y^\top \bar{A}^\top \bar{A}\, y + \tilde{c}^\top y \tag{3.5a}$$

$$\text{s.t.} \quad \bar{C}y = \mathbf{1}, \tag{3.5b}$$

$$y \geq 0, \tag{3.5c}$$

with the linear cost

$$\tilde{c} = \bar{c} - \bar{A}^\top\Big(\hat{\pi} + \frac{b}{\delta}\Big). \tag{3.6}$$

More precisely, (y, w) is optimal for (3.3) if and only if y is optimal for (3.5) and $w = (b - \bar{A}y)/\delta$. The objective values are related by

$$\Phi(y) := \bar{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2 = f(y) + \frac{1}{2\delta}\|b\|^2 + \hat{\pi}^\top b. \tag{3.7}$$

Proof. Let $r = b - \bar{A}y$, so $w = r/\delta$. Then

$$\tfrac{\delta}{2}\|r/\delta + \hat{\pi}\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 = \tfrac{1}{2\delta}\|r\|^2 + \hat{\pi}^\top r = \tfrac{1}{2\delta}\|\bar{A}y\|^2 - \tfrac{1}{\delta}b^\top \bar{A}y + \tfrac{1}{2\delta}\|b\|^2 + \hat{\pi}^\top b - \hat{\pi}^\top \bar{A}y.$$

Adding $\bar{c}^\top y$ and collecting the terms that are linear in y gives $f(y)$ plus the constant in (3.7). The constraint (3.3b) only defines w, so the remaining constraints are (3.5b) and (3.5c). □

The Hessian of f is $\frac{1}{\delta}\bar{A}^\top\bar{A}$, with the block structure

$$\frac{1}{\delta}\bar{A}^\top\bar{A} = \frac{1}{\delta}\begin{bmatrix} A_E^\top A_E + A_I^\top A_I & -A_I^\top \\ -A_I & I \end{bmatrix}. \tag{3.8}$$

It is positive semidefinite but in general singular: it has rank at most m, while the number of variables $n + m_I$ grows with every generated column. Problem (3.5) nevertheless has an optimal solution, because its feasible region is bounded in the λ part and the objective grows quadratically in s.

3.3.1 The dual vector as a function of the primal iterate

Relation (3.4) with $w = (b - \bar{A}y)/\delta$ suggests the following definition, which applies to *every* y, not only to the optimal one.

Definition 3.3.2 (Dual estimate)**.** For $y = [\lambda; s]$ the **dual estimate** is

$$\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A}y), \qquad \text{i.e.} \qquad \begin{aligned} \pi_E(y) &= \hat{\pi}_E + \tfrac{1}{\delta}(b_E - A_E\lambda), \\ \pi_I(y) &= \hat{\pi}_I + \tfrac{1}{\delta}(b_I - A_I\lambda + s). \end{aligned} \tag{3.9}$$

A surplus raises the dual of its row: an over-covered inequality row with a positive surplus has a larger π_i than it would have without the surplus. The next lemma is the central identity of this work.

Lemma 3.3.3 (Gradient–dual relation)**.** For every y,

$$\nabla f(y) = \bar{c} - \bar{A}^\top \pi(y), \qquad \text{i.e.} \qquad g_j := \frac{\partial f}{\partial \lambda_j} = c_j - a_j^\top \pi(y), \qquad \frac{\partial f}{\partial s_i} = \pi_i(y) \quad (i \in \{1, \dots, m_I\}). \tag{3.10}$$

Proof. $\nabla f(y) = \frac{1}{\delta}\bar{A}^\top\bar{A}y + \bar{c} - \bar{A}^\top\hat{\pi} - \frac{1}{\delta}\bar{A}^\top b = \bar{c} - \bar{A}^\top\big(\hat{\pi} + \frac{1}{\delta}(b - \bar{A}y)\big)$. The rows of $\bar{A}^\top$ belonging to λ_j are $a_j^\top$, and the rows belonging to s_i are $-e_i^\top$ (in the inequality part). □

Lemma 3.3.3 has two consequences that are used throughout this work.

- The gradient component of a column, $g_j = c_j - a_j^\top\pi$, is the **reduced cost of column j without the convexity dual**. The gradient component of a surplus is the dual of its row.
- The gradient, an $(n + m_I)$-vector whose length grows with every column, is determined by the m-vector $\pi(y)$. An implementation can store π and evaluate g_j only where it is needed.

Proposition 3.3.4 (Objective value)**.** For every y, with $\pi = \pi(y)$,

$$\Phi(y) = c^\top\lambda + \frac{\delta}{2}\|\pi\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2.$$

Proof. In (3.7), $\Phi(y) = \bar{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2$ with $w + \hat{\pi} = \pi(y)$, and $\bar{c}^\top y = c^\top\lambda$ because the surpluses have zero cost. □

Proposition 3.3.5 (Uniqueness of the dual)**.** The optimal value of $\bar{A}y$, and hence the optimal dual vector π, is the same for all optimal solutions of (3.5), even if the optimal y is not unique.

Proof. Let y_1, y_2 be optimal with $\bar{A}y_1 \neq \bar{A}y_2$. Their midpoint $\bar{y}$ is feasible. The linear part of f is affine, while $\|\bar{A}y\|^2$ is strictly convex as a function of $\bar{A}y$, so $f(\bar{y}) < \frac{1}{2}(f(y_1) + f(y_2))$, a contradiction.

□

Proposition 3.3.6 (Residual and dual distance)**.** For every y,

$$b - \bar{A}y = \delta(\pi(y) - \hat{\pi}).$$

In particular, y is feasible for the unified master problem (2.6) if and only if $\pi(y) = \hat{\pi}$.

Proof. This is (3.9) rearranged. □

3.4 Regularized Column Generation

Column generation with the regularized master problem follows the classical scheme of Algorithm 2.1, with two differences: the duals passed to the pricing problem are the stabilized duals $\pi(y)$, and the stability center has to be updated.

3.4.1 Center updates

Two rules for the center update are common.

- **Proximal point rule:** after every master problem, $\hat{\pi} \leftarrow \pi$. Every iteration is a step of the proximal point method on the current restricted dual problem.
- **Bundle rule:** $\hat{\pi} \leftarrow \pi$ only if the Lagrangian bound improves, $\theta(\pi) > \theta(\hat{\pi})$ (a *serious step*). Otherwise the center stays (a *null step*). This requires the Lagrangian bound at every iteration, i.e. all blocks must be priced every iteration.

3.4.2 Exact termination

In classical column generation, the pricing test alone decides optimality (Proposition 2.3.3). With regularization this is no longer true: when pricing finds no improving column, the regularized master problem may still be infeasible for the unregularized one, since $\bar{A}y \neq b$ as long as $\pi \neq \hat{\pi}$ (Proposition 3.3.6).

Theorem 3.4.1 (Exact termination)**.** Let $y = [\lambda; s]$ be optimal for the eliminated regularized master problem (3.5) over the columns $\mathcal{J}'$, with dual estimate $\pi = \pi(y)$ and convexity duals μ. If

(i) $\pi = \hat{\pi}$, and

(ii) the pricing problem (2.5) has a nonnegative optimal value for every block,

then λ, extended by zeros, is optimal for the master problem (2.3), and (π, μ) is optimal for its dual (2.4).

Proof. By Proposition 3.3.6, $\pi = \hat{\pi}$ gives $\bar{A}y = b$, i.e. $A_E\lambda = b_E$ and $A_I\lambda = b_I + s \geq b_I$, so λ is primal feasible. The optimality conditions of (3.5), which are derived in Theorem 4.2.1 below, state that every column of $\mathcal{J}'$ has a nonnegative reduced cost (with equality where $\lambda_j > 0$), that $\pi_I \geq 0$, and that $\pi_i = 0$ whenever $s_i > 0$. Condition (ii) extends the nonnegativity of the reduced costs to all columns. So (π, μ) is dual feasible, and complementary slackness holds for the columns, the zero weights outside $\mathcal{J}'$ and the inequality rows. □

If pricing finds no improving column while $\pi \neq \hat{\pi}$, the restricted master problem contains the right columns for the moment, but the proximal term still keeps π away from the restricted dual optimum. The algorithm then moves the center to π and solves the master problem again *without adding a column*. We call this a **center-only iteration**. It is one proximal point step on the current restricted model. Along the way, the duals move, and pricing may find improving columns again. A run of regularized column generation therefore consists of two kinds of iterations, which Section 8.4 studies in detail: iterations that add columns, and center-only iterations that drive π towards $\hat{\pi}$.

Proposition 3.4.2 (Accuracy of the stopping test)**.** In practice condition (i) of Theorem 3.4.1 is tested as

$$\|\pi - \hat{\pi}\|_\infty \;\leq\; \tau \max(1, \|\pi\|_\infty) \tag{3.11}$$

with a tolerance $\tau > 0$. When this test holds, the violation of the linking rows by λ is bounded by

$$\|b - \bar{A}y\|_\infty \;\leq\; \delta\,\tau \max(1, \|\pi\|_\infty).$$

Proof. This follows from Proposition 3.3.6. □

The accuracy of the final solution is thus relative to the size of the duals, and it scales with δ. The objective value Φ and the Lagrangian bound are much less affected by this residual than the cost $c^\top \lambda$ of a slightly infeasible λ, as the results in Section 8.3 show.

3.4.3 The Lagrangian bound with inequality rows

The Lagrangian bound of Proposition 2.3.4 requires $\pi_I \geq 0$. The duals of the regularized master problem satisfy this condition only up to the tolerance of the master solver, and a tiny negative entry would make the bound useless (Remark 2.3.5). The bound is therefore evaluated at the projection

$$\pi^+ = \big[\, \pi_E;\ \max(\pi_I, 0) \,\big],$$

which always satisfies the sign condition, so that $\theta(\pi^+) \leq z^\star$ holds by Proposition 2.3.4. The same projected vector is passed to the pricing problem. For problems with equality rows only, $\pi^+ = \pi$.

3.4.4 The algorithm

Algorithm 3.1: Regularized Column Generation

Input: initial columns $\mathcal{J}'$ (one per block), center $\hat{\pi}$, parameter $\delta > 0$, tolerances τ, ε_{rc}
repeat
  Solve the eliminated regularized master problem (3.5); obtain y, $\pi = \pi(y)$ and μ
  $\pi^+ \leftarrow [\pi_E; \max(\pi_I, 0)]$
  Price the blocks in random order with (π^+, μ); add the first column with $\bar{c}_j < -\varepsilon_{\text{rc}}$ to $\mathcal{J}'$
  if no column was added (all blocks priced) **then**
    $\theta \leftarrow \theta(\pi^+)$ ▷ valid lower bound on $z^\star$
    if $\|\pi - \hat{\pi}\|_\infty \leq \tau \max(1, \|\pi\|_\infty)$ **then**
      stop: λ is optimal (Theorem 3.4.1)
    end if
    $\hat{\pi} \leftarrow \pi$ ▷ center-only iteration
  else
    $\hat{\pi} \leftarrow \pi$ ▷ proximal point rule (bundle rule: only if θ improves)
  end if
until stopped

In the experiments of Chapter 8 we use the proximal point rule, $\delta = 1$, $\tau = 10^{-6}$ and $\varepsilon_{\text{rc}} = 10^{-6}$. The remaining chapters develop the method used to solve the master problem in the first line of the loop.

Chapter 4

An Active-Set Method for the Regularized Master Problem

This chapter sets up the **quadratic programming active-set method (QPAS)** for the eliminated regularized master problem. We first derive its optimality conditions and show that the Lagrange multipliers of the nonnegativity constraints are the reduced costs of the columns and the duals of the inequality rows. The optimality test of the master problem and the pricing test of column generation are therefore the same test.

We then recall the primal active-set method: it fixes a **working set** of variables at zero, minimizes over the remaining free variables, and adds or removes one constraint at a time. Every iteration requires the solution of a KKT system whose constraint matrix consists of the convexity rows and the rows of the working set.

The main result of this chapter is a **reordering theorem** for this constraint matrix. The columns of the regularized master problem are partitioned into five sets: representatives, working-set columns, nullspace columns, active surpluses and free surpluses. In this ordering the constraint matrix has a block structure with identity blocks, which is the basis of the explicit nullspace method of the next chapter.

4.1 The Master Problem in Simplified Notation

From now on we consider a single restricted master problem with n columns. To keep the notation light, we drop the explicit reference to the restricted index set and write

- $x \in \mathbb{R}^n$ for the column weights (the vector λ restricted to $\mathcal{J}'$),
- $A_E \in \mathbb{R}^{m_E \times n}$, $A_I \in \mathbb{R}^{m_I \times n}$, $A = [A_E; A_I]$ and $C \in \{0,1\}^{K \times n}$ for the corresponding columns of the linking and convexity matrices,
- $c \in \mathbb{R}^n$ for the costs of the columns, and $a_j \in \mathbb{R}^m$ for column j of A,
- $y = [x; s] \in \mathbb{R}^{n+m_I}$ for the unified variable.

The vector c always denotes the original column costs; the linear term of the quadratic objective is $\tilde{c}$ from (3.6). With this notation the eliminated regularized master problem (3.5) reads

$$\min_{y \in \mathbb{R}^{n+m_I}} \quad f(y) = \frac{1}{2\delta}\, y^\top \bar{A}^\top \bar{A}\, y + \tilde{c}^\top y \tag{4.1a}$$

$$\text{s.t.} \quad \bar{C} y = \mathbf{1}, \tag{4.1b}$$

$$y \geq 0, \tag{4.1c}$$

with

$$\bar{A} = \begin{bmatrix} A_E & 0 \\ A_I & -I \end{bmatrix} \in \mathbb{R}^{m \times (n+m_I)}, \qquad \bar{C} = \begin{bmatrix} C & 0 \end{bmatrix} \in \mathbb{R}^{K \times (n+m_I)}.$$

We write $g = \nabla f(y)$ for the gradient at the current iterate and, by Lemma 3.3.3,

$$g = \begin{bmatrix} g_x \\ g_s \end{bmatrix} = \begin{bmatrix} c - A^\top \pi \\ \pi_I \end{bmatrix}, \qquad \pi = \pi(y) = \hat{\pi} + \tfrac{1}{\delta}(b - \bar{A} y). \tag{4.2}$$

4.2 Optimality Conditions

Problem (4.1) is a convex quadratic program with linear constraints, so the KKT conditions are necessary and sufficient for optimality.

Theorem 4.2.1 (KKT conditions of the regularized master problem)**.** A feasible point $y = [x; s]$ of (4.1) is optimal if and only if there exist $\mu \in \mathbb{R}^K$ and $\nu \in \mathbb{R}^{n+m_I}$ with

$$g = \bar{C}^\top \mu + \nu, \qquad \nu \geq 0, \qquad \nu_\ell\, y_\ell = 0 \quad \text{for all } \ell.$$

Written per variable, with $\pi = \pi(y)$, these conditions are equivalent to:

(i) **reduced costs:** $\bar{c}_j(\pi, \mu) = c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0$ for every column j, with equality if $x_j > 0$;

(ii) **sign condition:** $\pi_i \geq 0$ for every inequality row i;

(iii) **slackness:** $\pi_i = 0$ for every inequality row i with $s_i > 0$.

The multipliers are $\nu_j = \bar{c}_j(\pi, \mu)$ for the columns and $\nu_{n+i} = \pi_i$ for the surpluses.

Proof. The first part is the KKT theorem for convex quadratic programs. For a column j, the stationarity condition reads $g_j = \mu_{\psi(j)} + \nu_j$, so $\nu_j = c_j - a_j^\top \pi - \mu_{\psi(j)} = \bar{c}_j(\pi, \mu)$ by (4.2). For surplus i the column of $\bar{C}$ is zero, so $\nu_{n+i} = g_{n+i} = \pi_i$. The conditions $\nu \geq 0$ and $\nu_\ell y_\ell = 0$ then become (i)–(iii). □

Remark 4.2.2 (The optimality test is the pricing test)**.** By Theorem 4.2.1, the multipliers of the nonnegativity constraints of the columns are their reduced costs, the same quantity that the pricing problem minimizes over the columns that have not been generated yet. Solving the regularized master problem to optimality establishes "reduced cost ≥ 0" for the columns inside the restricted problem, and pricing establishes it for the columns outside. The multipliers of the surpluses carry the sign conditions (2.4c) of the inequality rows. Together with $\pi = \hat{\pi}$, this is exactly the optimality certificate of Theorem 3.4.1.

Remark 4.2.3 (Matching tolerances)**.** The active-set method accepts a multiplier as nonnegative if $\nu_\ell \geq -\varepsilon$, and pricing accepts a column only if $\bar{c}_j < -\varepsilon_{\mathrm{rc}}$. Choosing $\varepsilon \ll \varepsilon_{\mathrm{rc}}$ (we use $\varepsilon = 10^{-10}\max(1, \max_j |c_j|)$ and $\varepsilon_{\mathrm{rc}} = 10^{-6}$) guarantees that pricing never returns a column that is already in the restricted master problem.

4.3 The Primal Active-Set Method

Definition 4.3.1 (Active set and working set)**.** The **active set** of a feasible point y is $\mathcal{A}(y) = \{\ell : y_\ell = 0\}$. A **working set** is a set $\mathcal{W} \subseteq \{1, \dots, n + m_I\}$ of indices that are treated as active, i.e. $y_\ell = 0$ for all $\ell \in \mathcal{W}$. We split it into the **column working set** $\mathcal{W}_x$ and the **surplus working set** $\mathcal{W}_s$:

$$\mathcal{W} = \mathcal{W}_x \cup \{\, n + i : i \in \mathcal{W}_s \,\}, \qquad \mathcal{W}_x \subseteq \{1, \dots, n\}, \quad \mathcal{W}_s \subseteq \{1, \dots, m_I\}.$$

A surplus $i \in \mathcal{W}_s$ is called **active**; a surplus $i \notin \mathcal{W}_s$ is called **free**.

Given a working set and a feasible iterate y_k with $y_\ell = 0$ on $\mathcal{W}$, the active-set method computes a search direction from the **equality-constrained subproblem**

$$\min_p \ \frac{1}{2\delta} p^\top \bar{A}^\top \bar{A} p + g^\top p \qquad \text{s.t.} \qquad \bar{C} p = 0, \qquad p_\ell = 0 \ \ (\ell \in \mathcal{W}), \tag{4.3}$$

which minimizes $f(y_k + p)$ over the face of the feasible region defined by the working set. To write its constraints compactly, let $W_x \in \mathbb{R}^{|\mathcal{W}_x| \times n}$ and $W_s \in \mathbb{R}^{|\mathcal{W}_s| \times m_I}$ be the selection matrices whose rows are the unit vectors $e_j^\top$, $j \in \mathcal{W}_x$, and $e_i^\top$, $i \in \mathcal{W}_s$.

Definition 4.3.2 (Constraint matrix of a working set)**.** The **constraint matrix associated with** $\mathcal{W}$ is

$$D_{\mathcal{W}} = \begin{bmatrix} C & 0 \\ W_x & 0 \\ 0 & W_s \end{bmatrix} \in \mathbb{R}^{(K + |\mathcal{W}_x| + |\mathcal{W}_s|) \times (n + m_I)}. \tag{4.4}$$

The constraints of (4.3) are $D_{\mathcal{W}}\, p = 0$.

The KKT conditions of (4.3) form the linear **KKT system**

$$\begin{bmatrix} \frac{1}{\delta}\bar{A}^\top \bar{A} & D_{\mathcal{W}}^\top \\ D_{\mathcal{W}} & 0 \end{bmatrix} \begin{bmatrix} p \\ -\mu \\ -\nu_x \\ -\nu_s \end{bmatrix} = \begin{bmatrix} -g \\ 0 \\ 0 \\ 0 \end{bmatrix}, \tag{4.5}$$

where $\mu \in \mathbb{R}^K$, $\nu_x \in \mathbb{R}^{|\mathcal{W}_x|}$ and $\nu_s \in \mathbb{R}^{|\mathcal{W}_s|}$ are the Lagrange multipliers of the convexity constraints and of the working-set constraints of the columns and of the surpluses. If $p = 0$ solves (4.5), the iterate minimizes f over its face, and we call it a **subspace minimizer**. The generic method is Algorithm 4.1.

Algorithm 4.1: Primal Active-Set Method for the Regularized Master Problem

Input: feasible y, working set $\mathcal{W}$ with $y_\ell = 0$ on $\mathcal{W}$
repeat
  Solve the KKT system (4.5) for p, μ, $\nu = (\nu_x, \nu_s)$
  if $p = 0$ **then**
    if $\nu_\ell \geq -\varepsilon$ for all $\ell \in \mathcal{W}$ **then**
      stop: y is optimal
    else
      $r \leftarrow \arg\min_{\ell \in \mathcal{W}} \nu_\ell$; $\mathcal{W} \leftarrow \mathcal{W} \setminus \{r\}$ ▷ reduce the working set
    end if
  else
    $\alpha \leftarrow \min\Big\{1, \min_{\ell \notin \mathcal{W},\, p_\ell < 0} -\dfrac{y_\ell}{p_\ell}\Big\}$ ▷ ratio test
    $y \leftarrow y + \alpha p$
    if $\alpha < 1$ **then**
      $q \leftarrow$ the index attaining the minimum; $\mathcal{W} \leftarrow \mathcal{W} \cup \{q\}$ ▷ expand the working set
    end if
  end if
until stopped

Under the usual nondegeneracy assumptions, every iteration either decreases f strictly or changes the working set in a way that cannot cycle, so the method terminates after finitely many iterations [15, 9]. Its cost is dominated by the solution of the KKT system (4.5), which has dimension $n + m_I + K + |\mathcal{W}|$ and grows with every generated column. The rest of this work shows how to avoid ever forming it. The nullspace method of Chapter 5 needs two properties:

Assumption 4.3.3. (i) $D_{\mathcal{W}}$ has full row rank;

(ii) the reduced Hessian $Z^\top \bar{A}^\top \bar{A} Z$ is positive definite, where the columns of Z form a basis of $\mathrm{Null}(D_{\mathcal{W}})$.

Property (i) holds for every working set that keeps one column per block free (Theorem 4.4.6 below). Property (ii) can fail, and Chapter 7 shows how the method guarantees it.

4.4 The Constraint Matrix and the Reordering Theorem

The structure of $D_{\mathcal{W}}$ is determined by the convexity matrix C, which has the column generation property, and by the zero columns of the surpluses, which do not. We first arrange the columns of the weights x, and then add the surpluses.

Definition 4.4.1 (Free columns)**.** The **free columns** are $\mathcal{F}_x = \{1, \ldots, n\} \setminus \mathcal{W}_x$.

Lemma 4.4.2. If y is feasible and $x_j = 0$ for all $j \in \mathcal{W}_x$, then every block has at least one free column, and the submatrix $C_{\mathcal{F}_x}$ of C has the column generation property.

Proof. For every block k, $\sum_{\psi(j)=k} x_j = 1$ and $x_j = 0$ on $\mathcal{W}_x$, so some column of block k lies in $\mathcal{F}_x$. The claim follows from Lemma 2.2.6. □

Definition 4.4.3 (Representatives)**.** A **list of representatives** is an ordered list $\mathcal{S} = [S_1, \ldots, S_K]$ of free columns with $\psi(S_k) = k$, one per block. The **representative of column** j is

$$\mathrm{rep}(j) = S_{\psi(j)}.$$

The remaining free columns form the **nullspace set** $\mathcal{N}_x = \mathcal{F}_x \setminus \mathcal{S}$.

The representatives exist by Lemma 4.4.2. Their columns in C are the unit vectors $e_1, \dots, e_K$, so $C_{\mathcal{S}} = I_K$.

Definition 4.4.4 (Extended ordering)**.** The **extended ordering** of the variables of (4.1) is the partition into the five sets

set	contents	status
$\mathcal{S}$	one representative column per block	free
$\mathcal{W}_x$	columns in the working set	$x_j = 0$ fixed
$\mathcal{N}_x$	the remaining free columns	free
$\mathcal{W}_s$	active surpluses	$s_i = 0$ fixed
$\mathcal{N}_s$	free surpluses	free

We write $[\,\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x \mid \mathcal{W}_s, \mathcal{N}_s\,]$ for this ordering.

Theorem 4.4.5 (Reordering theorem)**.** Let $\mathcal{W} = \mathcal{W}_x \cup \mathcal{W}_s$ be a working set such that every block has a free column, and let $[\,\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x \mid \mathcal{W}_s, \mathcal{N}_s\,]$ be an extended ordering. Let Q be the permutation matrix that orders the variables in this way. Then

$$D_{\mathcal{W}}\, Q = \begin{bmatrix} I_K & C_{\mathcal{W}_x} & C_{\mathcal{N}_x} & 0 & 0 \\ 0 & I_{|\mathcal{W}_x|} & 0 & 0 & 0 \\ 0 & 0 & 0 & I_{|\mathcal{W}_s|} & 0 \end{bmatrix}, \tag{4.6}$$

where $C_{\mathcal{W}_x}$ and $C_{\mathcal{N}_x}$ are the columns of C indexed by $\mathcal{W}_x$ and $\mathcal{N}_x$.

Proof. The permutation is block-diagonal, $Q = \operatorname{diag}(Q_x, Q_s)$: Q_x orders the column weights as $[\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x]$ and Q_s orders the surpluses as $[\mathcal{W}_s, \mathcal{N}_s]$. Apply it to the three block rows of (4.4).

- The convexity rows $[\,C \;\; 0\,]$ become $[\,C_{\mathcal{S}} \;\; C_{\mathcal{W}_x} \;\; C_{\mathcal{N}_x} \;\; 0 \;\; 0\,]$, and $C_{\mathcal{S}} = I_K$ by Definition 4.4.3.
- The rows $[\,W_x \;\; 0\,]$ select the columns in $\mathcal{W}_x$, which appear in the second position: $[\,0 \;\; I_{|\mathcal{W}_x|} \;\; 0 \;\; 0 \;\; 0\,]$.
- The rows $[\,0 \;\; W_s\,]$ select the surpluses in $\mathcal{W}_s$, which appear in the fourth position: $[\,0 \;\; 0 \;\; 0 \;\; I_{|\mathcal{W}_s|} \;\; 0\,]$.

□

Theorem 4.4.6 (Full row rank)**.** Under the assumptions of Theorem 4.4.5, $D_{\mathcal{W}}$ has full row rank,

$$\operatorname{rank}(D_{\mathcal{W}}) = K + |\mathcal{W}_x| + |\mathcal{W}_s|,$$

and $\dim \operatorname{Null}(D_{\mathcal{W}}) = |\mathcal{N}_x| + |\mathcal{N}_s|$.

Proof. In (4.6), the identity blocks I_K, $I_{|\mathcal{W}_x|}$ and $I_{|\mathcal{W}_s|}$ lie in disjoint rows and disjoint columns, so the rows are linearly independent. Permutations do not change the rank. The dimension of the nullspace follows from the rank–nullity theorem: $(n + m_I) - (K + |\mathcal{W}_x| + |\mathcal{W}_s|) = |\mathcal{N}_x| + |\mathcal{N}_s|$, because $n = K + |\mathcal{W}_x| + |\mathcal{N}_x|$ and $m_I = |\mathcal{W}_s| + |\mathcal{N}_s|$. □

Theorem 4.4.6 establishes property (i) of Assumption 4.3.3 for every working set the method can produce: the ratio test never removes the last free column of a block, since the representative of the block can only reach zero while some other column of the block grows (Lemma 6.2.2 below).

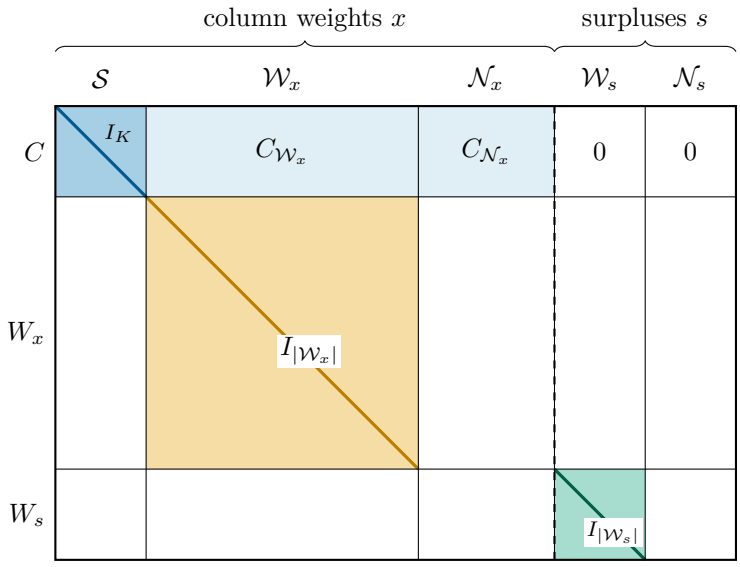


Figure 4.1: Block structure of the reordered constraint matrix $D_{\mathcal{W}}Q$ of Theorem 4.4.5. The diagonal lines are identity blocks. The surplus part (right of the dashed line) only contributes the identity block of the active surpluses; the free surpluses $\mathcal{N}_s$ have empty columns.

Remark 4.4.7 (Comparison with problems without inequality rows)**.** Without inequality rows ($m_I = 0$), the surplus sets are empty, and the extended ordering reduces to the three sets $[\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x]$. The inequality rows thus only add two groups of columns: active surpluses, which behave like working-set columns, and free surpluses, which behave like nullspace columns without a representative. The loss of the column generation property (Remark 2.4.3) does not affect the reordering. It only means that the surpluses form groups of their own.

Figure 4.2 shows the effect of the reordering on an actual constraint matrix, taken from the final working set of a column generation run on one of the test instances of Chapter 8.

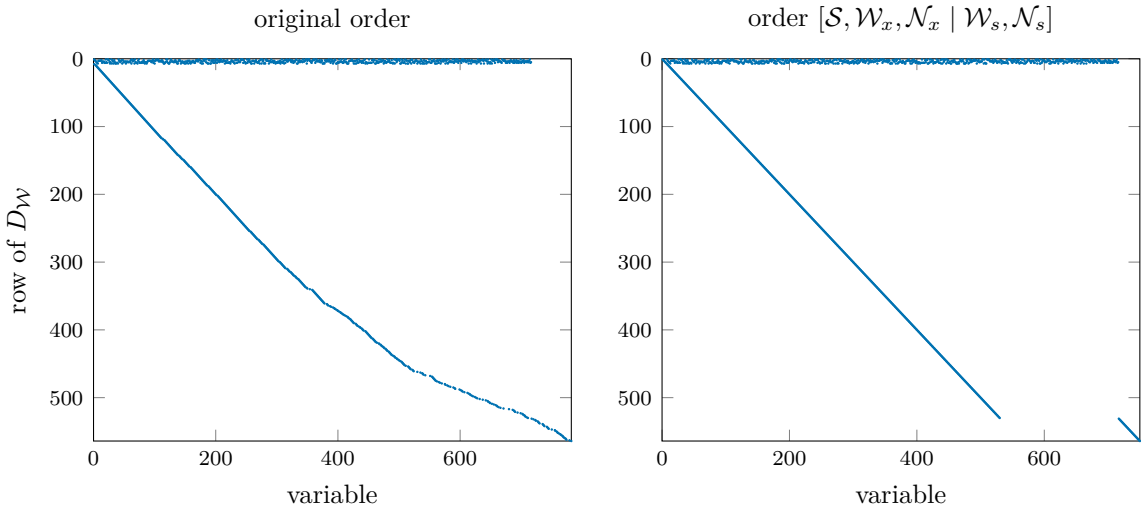


Figure 4.2: Sparsity pattern of the constraint matrix $D_{\mathcal{W}}$ at the end of a column generation run (instance I2 with 30% inequality rows), before and after the reordering of Theorem 4.4.5. After reordering, the identity blocks of the representatives (top left), the working-set columns (diagonal) and the active surpluses (bottom right) are visible.

Chapter 5

The Nullspace Method

The nullspace method solves the KKT system (4.5) in two smaller steps. The search direction is written as $p = Z\eta$, where the columns of Z span the nullspace of the constraint matrix. This turns the KKT system into a **reduced system** with the reduced Hessian, whose dimension equals the number of degrees of freedom of the current face. The Lagrange multipliers are then recovered from the remaining equations.

For the regularized master problem both steps can be carried out explicitly. The reordering theorem of the previous chapter gives a nullspace basis that consists of the vectors $e_j - e_{\mathrm{rep}(j)}$ and of unit vectors for the free surpluses. This basis is **never stored**: it is determined by the five sets alone.

The central result of this chapter is a **decoupling** of the reduced system. The surplus part of the direction is given by an explicit formula, and only an $|\mathcal{N}_x| \times |\mathcal{N}_x|$ system has to be solved. Its matrix is the **effective reduced Hessian** $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$, the reduced Hessian of the equality rows together with the inequality rows whose surplus is active. Finally, the multipliers turn out to be the reduced costs of the columns and the duals of the inequality rows, available at almost no cost.

5.1 The Nullspace Basis

By Theorem 4.4.6, the nullspace of $D_{\mathcal{W}}$ has dimension $|\mathcal{N}_x| + |\mathcal{N}_s|$. We label its basis vectors by the indices of these two sets.

Theorem 5.1.1 (Nullspace basis)**.** Under the assumptions of Theorem 4.4.5, the matrix

$$Z = Q\tilde{Z}, \qquad \tilde{Z} = \begin{bmatrix} -C_{\mathcal{N}_x} & 0 \\ 0 & 0 \\ I_{|\mathcal{N}_x|} & 0 \\ 0 & 0 \\ 0 & I_{|\mathcal{N}_s|} \end{bmatrix} \begin{matrix} \leftarrow \mathcal{S} \\ \leftarrow \mathcal{W}_x \\ \leftarrow \mathcal{N}_x \\ \leftarrow \mathcal{W}_s \\ \leftarrow \mathcal{N}_s \end{matrix} \tag{5.1}$$

is a basis of $\mathrm{Null}(D_{\mathcal{W}})$. In the original order of the variables it is block-diagonal,

$$Z = \begin{bmatrix} Z_x & 0 \\ 0 & Z_s \end{bmatrix}, \qquad Z_x \in \mathbb{R}^{n\times|\mathcal{N}_x|}, \quad Z_s \in \mathbb{R}^{m_I\times|\mathcal{N}_s|}, \tag{5.2}$$

where Z_x is a nullspace basis for the column weights and Z_s is the selection matrix of the free surpluses.

Proof. Write $\tilde{Z} = Q^\top Z$ in the five row blocks of the extended ordering,

$$\tilde{Z} = [\, Z_{\mathcal{S}};\ Z_{\mathcal{W}_x};\ Z_{\mathcal{N}_x};\ Z_{\mathcal{W}_s};\ Z_{\mathcal{N}_s} \,].$$

By (4.6), $D_{\mathcal{W}}Z = (D_{\mathcal{W}}Q)\tilde{Z} = 0$ is equivalent to

$$Z_{\mathcal{S}} + C_{\mathcal{W}_x}Z_{\mathcal{W}_x} + C_{\mathcal{N}_x}Z_{\mathcal{N}_x} = 0, \qquad Z_{\mathcal{W}_x} = 0, \qquad Z_{\mathcal{W}_s} = 0.$$

The blocks $Z_{\mathcal{N}_x}$ and $Z_{\mathcal{N}_s}$ are free, so the nullspace has the stated dimension, and the choice $Z_{\mathcal{N}_x} = I$, $Z_{\mathcal{N}_s} = I$ (with the corresponding zero columns) gives a basis with $Z_{\mathcal{S}} = -C_{\mathcal{N}_x}$ in the first group of columns and $Z_{\mathcal{S}} = 0$ in the second. The column weights and the surpluses do not mix, which gives (5.2). □

Corollary 5.1.2 (Explicit nullspace vectors)**.** The columns of Z are

$$z_j = e_j - e_{\mathrm{rep}(j)} \quad (j \in \mathcal{N}_x), \qquad\qquad z_i = e_{n+i} \quad (i \in \mathcal{N}_s), \tag{5.3}$$

where e_ℓ denotes the ℓ-th unit vector in $\mathbb{R}^{n+m_I}$.

Proof. Column $j \in \mathcal{N}_x$ of $\tilde{Z}$ has a 1 in the position of j and the entries of $-C_{\mathcal{N}_x}e_j = -e_{\psi(j)}$ in the representative block. By the column generation property, this is a single -1 in the row of block $\psi(j)$, i.e. at the position of the representative $S_{\psi(j)} = \mathrm{rep}(j)$. Column $i \in \mathcal{N}_s$ has a single 1 at the position of surplus i. □

Corollary 5.1.2 shows that Z is completely determined by the extended ordering: by the sets $\mathcal{N}_x$, $\mathcal{N}_s$ and the representative mapping. It therefore never has to be formed. The same holds for the product of the linking matrix with the nullspace vectors,

$$\bar{A}z_j = \begin{bmatrix} a^E_j - a^E_{\mathrm{rep}(j)} \\ a^I_j - a^I_{\mathrm{rep}(j)} \end{bmatrix} = a_j - a_{\mathrm{rep}(j)} =: u_j \quad (j \in \mathcal{N}_x), \qquad \bar{A}z_i = -e^{(I)}_i \quad (i \in \mathcal{N}_s), \tag{5.4}$$

where $e^{(I)}_i \in \mathbb{R}^m$ is the unit vector of inequality row i.

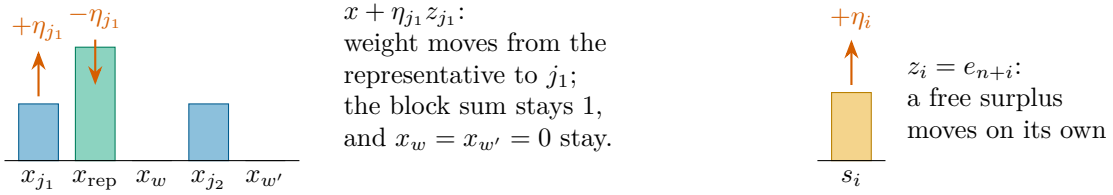


Figure 5.1: Interpretation of the nullspace vectors (5.3). A vector $z_j = e_j - e_{\text{rep}(j)}$ transfers weight between a nullspace column and the representative of its block, which keeps the convexity constraint satisfied and the working-set columns at zero. A free surplus belongs to no block and moves independently.

5.2 The Reduced KKT System

5.2.1 The reduced system

Every direction that satisfies the constraints of (4.3) has the form $p = Z\eta$ with $\eta = [\eta_x; \eta_s] \in \mathbb{R}^{|\mathcal{N}_x|+|\mathcal{N}_s|}$. Substituting this into the first block row of (4.5) and multiplying by $Z^\top$, which annihilates $D_{\mathcal{W}}^\top$, gives the **reduced KKT system**

$$\frac{1}{\delta}\, Z^\top \bar{A}^\top \bar{A} Z\, \eta = -Z^\top g. \tag{5.5}$$

We call $\bar{H} = Z^\top \bar{A}^\top \bar{A} Z$ the (unscaled) **reduced Hessian**. The vector η minimizes $f(y_k + Z\eta) = f(y_k) + (Z^\top g)^\top \eta + \frac{1}{2\delta}\eta^\top \bar{H}\eta$, so (5.5) is the subproblem (4.3) expressed in the coordinates of the nullspace. The parameter δ only appears on the right-hand side if we write the system as $\bar{H}\eta = -\delta Z^\top g$.

5.2.2 Block structure

To exhibit the structure of $\bar{H}$, we introduce the products of the linking matrices with the column part of the basis,

$$B_E := A_E Z_x \in \mathbb{R}^{m_E \times |\mathcal{N}_x|}, \qquad B_I := A_I Z_x \in \mathbb{R}^{m_I \times |\mathcal{N}_x|},$$

and split the rows of B_I and of π_I according to the status of the surplus of each row:

$$B_{\mathcal{W}} := W_s B_I, \quad \pi_{\mathcal{W}} := W_s \pi_I \qquad \text{(rows with an active surplus)},$$
$$B_{\mathcal{N}} := Z_s^\top B_I, \quad \pi_{\mathcal{N}} := Z_s^\top \pi_I \qquad \text{(rows with a free surplus)}.$$

Since every inequality row belongs to exactly one of the two groups, $W_s^\top W_s + Z_s Z_s^\top = I$ and therefore

$$B_I^\top B_I = B_{\mathcal{W}}^\top B_{\mathcal{W}} + B_{\mathcal{N}}^\top B_{\mathcal{N}}. \tag{5.6}$$

Lemma 5.2.1 (Blocks of the reduced system)**.** With (5.2),

$$\bar{A}Z = \begin{bmatrix} B_E & 0 \\ B_I & -Z_s \end{bmatrix}, \qquad \bar{H} = \begin{bmatrix} H & -B_{\mathcal{N}}^\top \\ -B_{\mathcal{N}} & I \end{bmatrix}, \qquad Z^\top g = \begin{bmatrix} Z_x^\top g_x \\ \pi_{\mathcal{N}} \end{bmatrix},$$

where $H = B_E^\top B_E + B_I^\top B_I = (AZ_x)^\top (AZ_x)$ is the reduced Hessian of the column weights with all linking rows.

Proof. The first identity follows from the block forms of $\bar{A}$ and Z. Then $\bar{H} = (\bar{A}Z)^\top(\bar{A}Z)$ has the blocks $B_E^\top B_E + B_I^\top B_I$, $-B_I^\top Z_s = -B_{\mathcal{N}}^\top$ and $Z_s^\top Z_s = I$. Finally $Z^\top g = [Z_x^\top g_x;\, Z_s^\top g_s]$ and $g_s = \pi_I$ by (4.2).

□

5.2.3 Decoupling and the effective reduced Hessian

The identity block in the lower right corner of $\bar{H}$ allows us to eliminate the surplus part of the direction.

Theorem 5.2.2 (Decoupling of the reduced system)**.** The reduced system (5.5) is equivalent to

$$H_{\text{eff}}\,\eta_x = -\delta\, Z_x^\top (c - A_E^\top \pi_E - A_I^\top W_s^\top \pi_{\mathcal{W}}), \tag{5.7}$$

$$\eta_s = B_{\mathcal{N}}\,\eta_x - \delta\,\pi_{\mathcal{N}}, \tag{5.8}$$

with the **effective reduced Hessian**

$$H_{\text{eff}} := H - B_{\mathcal{N}}^\top B_{\mathcal{N}} = B_E^\top B_E + B_{\mathcal{W}}^\top B_{\mathcal{W}}. \tag{5.9}$$

Proof. Multiply (5.5) by δ and write it in the blocks of Lemma 5.2.1:

$$H\eta_x - B_{\mathcal{N}}^\top \eta_s = -\delta Z_x^\top g_x, \qquad -B_{\mathcal{N}}\eta_x + \eta_s = -\delta\pi_{\mathcal{N}}.$$

The second equation is (5.8). Substituting it into the first gives $(H - B_{\mathcal{N}}^\top B_{\mathcal{N}})\eta_x = -\delta(Z_x^\top g_x + B_{\mathcal{N}}^\top \pi_{\mathcal{N}})$. The second expression for H_{eff} follows from (5.6). For the right-hand side, insert $g_x = c - A_E^\top \pi_E - A_I^\top \pi_I$ and split $Z_x^\top A_I^\top \pi_I = B_I^\top \pi_I = B_{\mathcal{W}}^\top \pi_{\mathcal{W}} + B_{\mathcal{N}}^\top \pi_{\mathcal{N}}$. The terms with $\pi_{\mathcal{N}}$ cancel, and $B_{\mathcal{W}}^\top \pi_{\mathcal{W}} = Z_x^\top A_I^\top W_s^\top \pi_{\mathcal{W}}$.

□

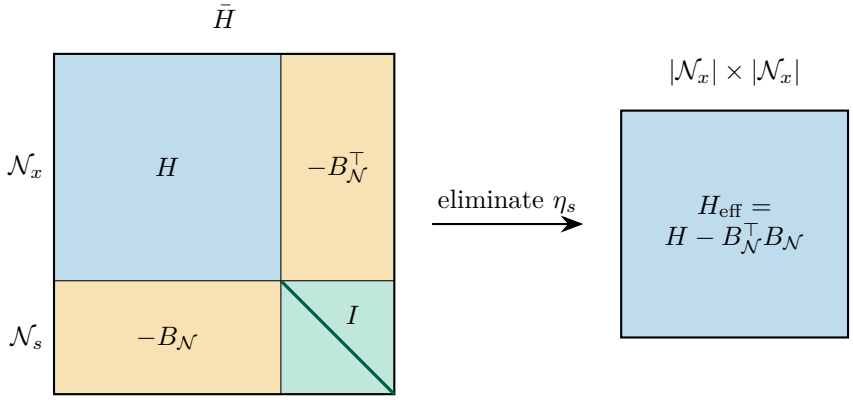


Figure 5.2: The reduced Hessian $\bar{H}$ of Lemma 5.2.1 and its Schur complement. Eliminating the identity block of the free surpluses leaves the effective reduced Hessian H_{eff} on the nullspace columns only.

H_{eff} is the Schur complement of the identity block in $\bar{H}$ (Fig. 5.2). Only an $|\mathcal{N}_x| \times |\mathcal{N}_x|$ system has to be solved; the surplus part (5.8) costs one sparse matrix–vector product. The following corollary gives the interpretation that the rest of this work builds on.

Corollary 5.2.3 (Effective linking matrix)**.** Define the **effective linking matrix** and the corresponding duals

$$A_{\text{eff}} = \begin{bmatrix} A_E \\ W_s A_I \end{bmatrix}, \qquad \pi_{\text{eff}} = \begin{bmatrix} \pi_E \\ \pi_{\mathcal{W}} \end{bmatrix},$$

i.e. the equality rows together with the inequality rows whose surplus is active. Then

$$H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x), \qquad H_{\text{eff}}\,\eta_x = -\delta\, Z_x^\top (c - A_{\text{eff}}^\top \pi_{\text{eff}}). \tag{5.10}$$

This is the reduced system of the master problem with equality linking rows only, whose linking rows are the rows of A_{eff}.

Proof. $A_{\text{eff}} Z_x = [B_E; B_{\mathcal{W}}]$, which gives the first identity by (5.9). The second is the right-hand side of (5.7), since $A_{\text{eff}}^\top \pi_{\text{eff}} = A_E^\top \pi_E + A_I^\top W_s^\top \pi_{\mathcal{W}}$. □

Remark 5.2.4 (Interpretation)**.** An inequality row with a *free* surplus does not restrict the column weights: any change of its residual is absorbed by the surplus, so the row contributes neither curvature nor dual pressure to the column part of the problem. An inequality row with an *active* surplus behaves like an equality row. The column part of the method is therefore the method for equality rows, applied to a linking matrix A_{eff} whose rows change with the surplus working set. Without inequality rows, or with all surpluses active, $A_{\text{eff}} = A$ and $H_{\text{eff}} = H$.

Corollary 5.2.5 (Positive definiteness and dimension)**.** $\bar{H}$ is positive definite if and only if H_{eff} is, i.e. if and only if $A_{\text{eff}} Z_x$ has full column rank. In this case

$$|\mathcal{N}_x| \;\le\; \operatorname{rank}(A_{\text{eff}}) \;\le\; m_E + |\mathcal{W}_s|, \qquad \text{and hence} \qquad |\mathcal{N}_x| + |\mathcal{N}_s| \le m.$$

Proof. A symmetric block matrix with a positive definite diagonal block is positive definite if and only if the Schur complement of that block is (Haynsworth inertia additivity). The rank bound holds because $H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x)$ is nonsingular only if $|\mathcal{N}_x| \le \operatorname{rank}(A_{\text{eff}} Z_x) \le \operatorname{rank}(A_{\text{eff}})$, and A_{eff} has $m_E + |\mathcal{W}_s|$ rows. Adding $|\mathcal{N}_s| = m_I - |\mathcal{W}_s|$ gives the last bound. □

Corollary 5.2.5 is the reason the method scales: the system that has to be factorized never exceeds the number of linking rows, *however many columns are generated.*

5.2.4 The search direction

With η from Theorem 5.2.2, the direction $p = Z\eta$ has the components

$$p_j = \eta_j \;(j \in \mathcal{N}_x), \qquad p_{S_k} = -\sum_{j \in \mathcal{N}_x,\, \psi(j)=k} \eta_j \;(k = 1, \dots, K), \qquad (p_s)_i = (A_I p_x)_i - \delta\, \pi_i \;(i \in \mathcal{N}_s), \tag{5.11}$$

and zero on $\mathcal{W}_x$ and $\mathcal{W}_s$. The surplus component uses $B_{\mathcal{N}} \eta_x = Z_s^\top A_I Z_x \eta_x = Z_s^\top A_I p_x$. The direction is supported on $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$, and its column part and the vector $A_I p_x$ only involve the columns in $\mathcal{S} \cup \mathcal{N}_x$.

Proposition 5.2.6 (Full step)**.** If the ratio test returns $\alpha = 1$, the new iterate $y_k + p$ is a subspace minimizer, i.e. the next reduced gradient $Z^\top \nabla f(y_k + p)$ vanishes.

Proof. On the affine set $\{y_k + Z\eta\}$, f is the convex quadratic $\phi(\eta) = f(y_k) + (Z^\top g)^\top \eta + \frac{1}{2\delta} \eta^\top \bar{H} \eta$, whose minimizer is the solution of (5.5). □

After a full step the method can therefore skip the next direction computation and test the multipliers directly. The next proposition shows how the free surpluses act on the duals.

Proposition 5.2.7 (Duals of free-surplus rows)**.** A step $y \leftarrow y + \alpha p$ changes the dual of every row with a free surplus as

$$\pi_i \;\leftarrow\; (1 - \alpha)\, \pi_i \qquad (i \in \mathcal{N}_s).$$

In particular $\pi_i = 0$ on these rows at every subspace minimizer.

Proof. By (3.9), π changes by $-\frac{\alpha}{\delta} \bar{A} p$. For row $i \in \mathcal{N}_s$, $(\bar{A}p)_i = (A_I p_x)_i - (p_s)_i = \delta \pi_i$ by (5.11). □

Remark 5.2.8 (Independence of δ and $\hat{\pi}$)**.** H_{eff} depends only on the columns and the working set, not on δ, $\hat{\pi}$ or the iterate. A change of the stability center or of δ only changes the right-hand side of (5.7). This is what makes the hot start of Section 6.6 possible.

5.2.5 Implicit evaluation

Neither Z, nor $A_{\text{eff}}Z_x$, nor the gradient has to be stored. With $u_j = a_j - a_{\text{rep}(j)}$ from (5.4) and the diagonal **row mask** $D_\rho \in \mathbb{R}^{m\times m}$ with $\rho_\ell = 0$ for the rows of free surpluses and $\rho_\ell = 1$ otherwise:

$$(H_{\text{eff}})_{kj} = u_k^\top D_\rho\, u_j, \qquad \left(Z_x^\top (c - A_{\text{eff}}^\top \pi_{\text{eff}})\right)_j = (c_j - a_j^\top D_\rho \pi) - (c_{\text{rep}(j)} - a_{\text{rep}(j)}^\top D_\rho \pi). \tag{5.12}$$

Both are computed from the sparse columns a_j and the m-vector π. The cost of one entry of the right-hand side is proportional to the number of nonzeros of two columns, independent of the total number of columns n.

5.3 The Lagrange Multipliers

The multipliers follow from the first block row of the KKT system (4.5):

$$D_{\mathcal{W}}^\top \begin{bmatrix} \mu \\ \nu_x \\ \nu_s \end{bmatrix} = h, \qquad h := \frac{1}{\delta}\bar{A}^\top \bar{A}\, p + g. \tag{5.13}$$

Theorem 5.3.1 (Lagrange multipliers)**.** With the extended ordering of Theorem 4.4.5 and $h = [h_{\mathcal{S}}; h_{\mathcal{W}_x}; h_{\mathcal{N}_x}; h_{\mathcal{W}_s}; h_{\mathcal{N}_s}]$, the solution of (5.13) is

$$\mu = h_{\mathcal{S}}, \qquad \nu_x = h_{\mathcal{W}_x} - C_{\mathcal{W}_x}^\top \mu, \qquad \nu_s = h_{\mathcal{W}_s}, \tag{5.14}$$

and moreover $h_{\mathcal{N}_x} = C_{\mathcal{N}_x}^\top \mu$ and $h_{\mathcal{N}_s} = 0$.

Proof. Multiply (5.13) by $Q^\top$ and use the transpose of (4.6):

$$\begin{bmatrix} I_K & 0 & 0 \\ C_{\mathcal{W}_x}^\top & I & 0 \\ C_{\mathcal{N}_x}^\top & 0 & 0 \\ 0 & 0 & I \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \mu \\ \nu_x \\ \nu_s \end{bmatrix} = \begin{bmatrix} h_{\mathcal{S}} \\ h_{\mathcal{W}_x} \\ h_{\mathcal{N}_x} \\ h_{\mathcal{W}_s} \\ h_{\mathcal{N}_s} \end{bmatrix}.$$

The first, second and fourth block rows give (5.14); the third and fifth are the remaining identities. □

The multipliers are only needed at a subspace minimizer, where $p = 0$ and therefore $h = g$. In this case Theorem 5.3.1 takes a particularly simple form.

Corollary 5.3.2 (Multipliers at a subspace minimizer)**.** At a subspace minimizer, with $\pi = \pi(y)$,

$$\begin{aligned} \mu_k &= c_{S_k} - a_{S_k}^\top \pi && (k = 1, \dots, K), \\ (\nu_x)_j &= c_j - a_j^\top \pi - \mu_{\psi(j)} = \bar{c}_j(\pi, \mu) && (j \in \mathcal{W}_x), \\ (\nu_s)_i &= \pi_i && (i \in \mathcal{W}_s), \end{aligned}$$

and $\bar{c}_j(\pi, \mu) = 0$ for $j \in \mathcal{S} \cup \mathcal{N}_x$, $\pi_i = 0$ for $i \in \mathcal{N}_s$.

Proof. Insert $h = g$ and (4.2) into Theorem 5.3.1. For $j \in \mathcal{N}_x$, $h_{\mathcal{N}_x} = C_{\mathcal{N}_x}^\top \mu$ reads $g_j = \mu_{\psi(j)}$, i.e. $\bar{c}_j = 0$, and $h_{\mathcal{N}_s} = 0$ reads $\pi_i = 0$. □

Corollary 5.3.2 gives a concrete form of Remark 4.2.2: the optimality test of the active-set method checks the reduced costs of the working-set columns and the signs of the duals of the inequality rows with active surpluses. All of them are computed from the single vector π. The test touches every column of $\mathcal{W}_x$ once and costs $O(\mathrm{nnz}(A))$; it is the only operation of the method whose cost grows with the number of generated columns.

Remark 5.3.3 ("$\leq$" rows)**.** For a "$\leq$" row with slack coefficient $\kappa_i = +1$ (Remark 2.4.4), the formulas above change as follows; everything else, in particular H_{eff} and A_{eff}, is unchanged because $\kappa_i^2 = 1$:

$$\pi_I = \hat{\pi}_I + \tfrac{1}{\delta}(b_I - A_I x - K_s s), \qquad g_{n+i} = -\kappa_i \pi_i, \qquad (\nu_s)_i = -\kappa_i \pi_i, \qquad (p_s)_i = \kappa_i(\delta \pi_i - (A_I p_x)_i).$$

For $\kappa_i = -1$ these are the formulas of the text.

5.4 Summary of One Iteration

Algorithm 5.1 collects the computations of one iteration of the active-set method in nullspace form, and Table 5.1 lists their costs. Here $\bar{\nu}$ denotes the average number of nonzeros of a column a_j.

Algorithm 5.1: One Iteration of the Active-Set Method in Nullspace Form

Given: iterate $y = [x; s]$, the extended ordering, $\pi = \pi(y)$, the factor $H_{\text{eff}} = LDL^\top$
Compute $\mu_k^\rho = c_{S_k} - a_{S_k}^\top D_\rho \pi$ and the right-hand side $r_j = -\delta\big(c_j - a_j^\top D_\rho \pi - \mu^\rho_{\psi(j)}\big)$, $j \in \mathcal{N}_x$
Solve $LDL^\top \eta_x = r$
Expand $p_x = Z_x \eta_x$ and compute $(p_s)_i = (A_I p_x)_i - \delta \pi_i$, $i \in \mathcal{N}_s$
if $p \approx 0$ **then**
    $\mu_k = c_{S_k} - a_{S_k}^\top \pi$; $\nu_x =$ reduced costs on $\mathcal{W}_x$; $\nu_s = \pi$ on $\mathcal{W}_s$
    Stop if all multipliers are $\geq -\varepsilon$; otherwise reduce the working set (Section 6.3)
else
    Ratio test over $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$; $y \leftarrow y + \alpha p$; $\pi \leftarrow \pi - \frac{\alpha}{\delta} \bar{A} p$
    If blocked, expand the working set (Section 6.2)
end if

operation	cost	grows with n?
right-hand side of (5.7)	$O\big((\lvert\mathcal{N}_x\rvert + K)\,\bar{\nu}\big)$	no
solve with the factor of H_{eff}	$O(\mathrm{nnz}(L))$, at most $O(\lvert\mathcal{N}_x\rvert^2)$	no
surplus direction (5.8)	$O\big((\lvert\mathcal{N}_x\rvert + K)\,\bar{\nu} + m_I\big)$	no
ratio test, step and update of π	$O\big((\lvert\mathcal{N}_x\rvert + K)\,\bar{\nu} + m\big)$	no
multipliers (Corollary 5.3.2)	$O(\mathrm{nnz}(A) + m_I)$	yes

Table 5.1: Cost of the operations of one iteration. Only the multiplier test touches all columns.

Chapter 6

Efficient Updates of the KKT System

Every iteration of the active-set method changes the working set by exactly one index: a blocking variable is added, or a variable with a negative multiplier is released. The previous chapter showed that everything the method needs is determined by the extended ordering and by a factorization of the effective reduced Hessian H_{eff}. This chapter shows that a change of the working set changes H_{eff} only by a **low-rank modification**, so that its $LDL^\top$ factorization can be updated in place instead of being recomputed.

We first introduce a **fixed-length representation**, in which the factorization has a fixed dimension and inactive indices are represented by identity rows. We then derive the updates for the three ways of **expanding** the working set and the two ways of **reducing** it, and translate them into row additions, row deletions and rank-one modifications of the factorization. Row deletions leave structural zeros in the factor; we analyse this effect and the **periodic refactorization** that controls it.

Finally, we show that the state of the method can be carried over from one column generation iteration to the next. A new column and a new stability center leave the factorization unchanged, which gives a **hot start** of every master problem. We also analyse the center-only iterations of Section 3.4 in this light.

6.1 Fixed-Length Representation

The number of nullspace columns $|\mathcal{N}_x|$ changes in almost every iteration, and the number of columns n grows during column generation. Resizing a sparse factorization at every change would be expensive and complicated. Instead, all objects are given a fixed dimension.

Definition 6.1.1 (Fixed-length representation)**.** Let $n_{\max} \geq n$ be a fixed **column capacity**, the maximal number of columns that column generation may generate. The **embedded effective reduced Hessian** is the $n_{\max} \times n_{\max}$ matrix

$$(H_{\text{eff}}^{\text{emb}})_{kj} = \begin{cases} (H_{\text{eff}})_{kj} = u_k^\top D_\rho u_j, & k, j \in \mathcal{N}_x, \\ 1, & k = j \notin \mathcal{N}_x, \\ 0, & \text{otherwise}, \end{cases}$$

i.e. index j of the embedded matrix is column j of the master problem. Indices of columns that are not in $\mathcal{N}_x$, including columns that have not been generated yet, carry an identity row and column.

In this representation, a column entering $\mathcal{N}_x$ replaces an identity row by a row of H_{eff}, and a column leaving $\mathcal{N}_x$ is replaced by an identity row. The dimension never changes. The surpluses do not appear at all: by Theorem 5.2.2 they have no index in the factorized matrix, so the inequality rows do not increase the size of the factorization.

Lemma 6.1.2 (Decoupling of inactive indices)**.** Let $H_{\text{eff}}^{\text{emb}} = LDL^\top$ be the $LDL^\top$ factorization without pivoting. Then for every $k \notin \mathcal{N}_x$, row and column k of L are e_k and $D_{kk} = 1$. Consequently, for a right-hand side r that vanishes outside $\mathcal{N}_x$, the solution of $H_{\text{eff}}^{\text{emb}}\eta = r$ vanishes outside $\mathcal{N}_x$, and its restriction to $\mathcal{N}_x$ solves $H_{\text{eff}}\eta_x = r_{\mathcal{N}_x}$.

Proof. After a symmetric permutation that moves the indices of $\mathcal{N}_x$ to the front, $H_{\text{eff}}^{\text{emb}} = \operatorname{diag}(H_{\text{eff}}, I)$. The $LDL^\top$ factorization of a block-diagonal matrix is block-diagonal, and that of I is $L = D = I$. Since the factorization without pivoting is unique and symmetric permutations only reorder it, the same holds in the natural order. The second statement follows from the block-diagonal structure. □

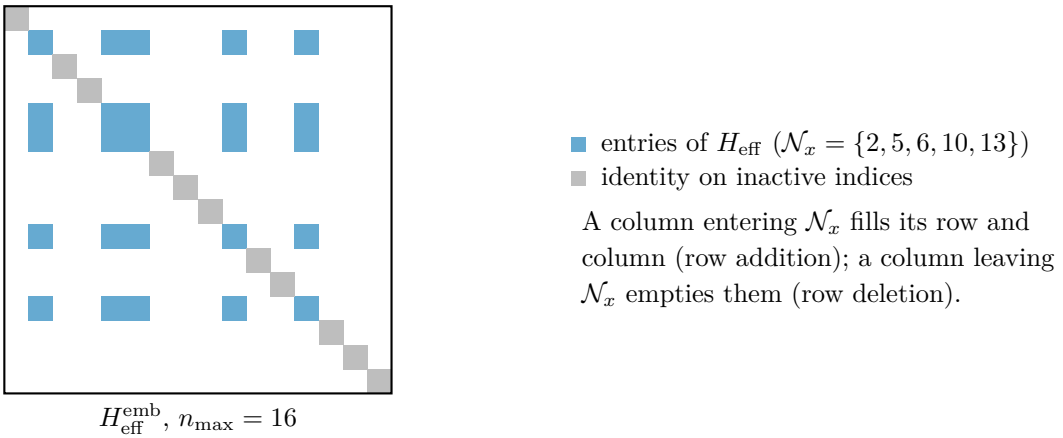


Figure 6.1: Fixed-length representation of the effective reduced Hessian: the active indices $\mathcal{N}_x$ are scattered over the fixed index range $1, \ldots, n_{\max}$, and every other index carries an identity row.

6.2 Expanding the Working Set

A step is blocked when a free variable reaches zero. The blocking index q is then added to the working set. There are three cases, depending on the set that q belongs to.

Lemma 6.2.1 (Ordering after an expansion)**.** Let q be the blocking index. The extended ordering is updated as follows.

(i) $q \in \mathcal{N}_x$: $\mathcal{N}_x \leftarrow \mathcal{N}_x \setminus \{q\}$, $\mathcal{W}_x \leftarrow \mathcal{W}_x \cup \{q\}$.

(ii) $q = S_k \in \mathcal{S}$: choose a **replacement** $r \in \nu_k := \{j \in \mathcal{N}_x : \psi(j) = k\}$; then $S_k \leftarrow r$, $\mathcal{N}_x \leftarrow \mathcal{N}_x \setminus \{r\}$, $\mathcal{W}_x \leftarrow \mathcal{W}_x \cup \{q\}$.

(iii) q is the free surplus $i \in \mathcal{N}_s$: $\mathcal{N}_s \leftarrow \mathcal{N}_s \setminus \{i\}$, $\mathcal{W}_s \leftarrow \mathcal{W}_s \cup \{i\}$.

Lemma 6.2.2 (Existence of a replacement)**.** In case (ii), the set ν_k is not empty.

Proof. The representative S_k can only decrease along the direction if $p_{S_k} = -\sum_{j\in\nu_k} \eta_j < 0$ by (5.11), which requires $\nu_k \neq \emptyset$. □

Any $r \in \nu_k$ is a valid replacement. A good choice is the column with the largest weight, $r = \arg\max_{j\in\nu_k} x_j$, since it is the least likely to block in the next step.

Theorem 6.2.3 (Update of the nullspace basis)**.** After an expansion, the nullspace basis changes as follows.

(i) $q \in \mathcal{N}_x$: the vector z_q is removed.

(ii) $q = S_k$ with replacement r: the vector z_r is removed, and for every $j \in \nu_k \setminus \{r\}$

$$z'_j = e_j - e_r = z_j - z_r.$$

All other vectors are unchanged.

(iii) q the surplus i: the vector e_{n+i} is removed from Z_s.

Proof. Cases (i) and (iii) follow from Corollary 5.1.2, since the index leaves $\mathcal{N}_x$ or $\mathcal{N}_s$. In case (ii), the representative of block k changes from q to r, so for $j \in \nu_k \setminus \{r\}$, $z'_j = e_j - e_r = (e_j - e_q) - (e_r - e_q) = z_j - z_r$.

□

Theorem 6.2.4 (Update of the effective reduced Hessian, expansion)**.** After an expansion, H_{eff} changes as follows.

(i) $q \in \mathcal{N}_x$: row and column q are removed.

(ii) $q = S_k$ with replacement r: row and column r are removed, and for $i, j \in \nu_k \setminus \{r\}$ and $\ell \notin \nu_k$

$$(H'_{\text{eff}})_{\ell j} = (H_{\text{eff}})_{\ell j} - (H_{\text{eff}})_{\ell r}, \qquad (H'_{\text{eff}})_{ij} = (H_{\text{eff}})_{ij} - (H_{\text{eff}})_{ir} - (H_{\text{eff}})_{rj} + (H_{\text{eff}})_{rr}.$$

(iii) q the surplus of inequality row i: row i joins A_{eff}, and

$$H'_{\text{eff}} = H_{\text{eff}} + v_i v_i^\top, \qquad v_i = Z_x^\top (A_I)_{i,:}^\top, \qquad (v_i)_j = (A_I)_{ij} - (A_I)_{i,\text{rep}(j)} \ \ (j \in \mathcal{N}_x). \tag{6.1}$$

Proof. (i) The column $A_{\text{eff}} z_q$ is removed from $A_{\text{eff}} Z_x$. (ii) The columns of $A_{\text{eff}} Z_x$ change by Theorem 6.2.3, $A_{\text{eff}} z'_j = A_{\text{eff}} z_j - A_{\text{eff}} z_r$; inserting this into $(H_{\text{eff}})_{\ell j} = (A_{\text{eff}} z_\ell)^\top (A_{\text{eff}} z_j)$ gives the formulas. (iii) The nullspace columns are unchanged, and A_{eff} gains the row $(A_I)_{i,:}$. Hence $A'_{\text{eff}} Z_x = [A_{\text{eff}} Z_x;\ (A_I)_{i,:} Z_x]$ and $H'_{\text{eff}} = H_{\text{eff}} + Z_x^\top (A_I)_{i,:}^\top (A_I)_{i,:} Z_x$. □

The vector v_i is row i of $B_I = A_I Z_x$. Its entry $(v_i)_j$ is nonzero only if exactly one of the columns j and $\text{rep}(j)$ has a nonzero coefficient in row i; in crew scheduling, if exactly one of the two schedules covers flight i.

Proposition 6.2.5. An expansion preserves positive definiteness of H_{eff}.

Proof. In cases (i) and (ii), $A_{\text{eff}} Z_x$ loses a column or is multiplied from the right by a nonsingular matrix (a change of basis of its column space) and then loses a column; full column rank is preserved in both cases. In case (iii), $H_{\text{eff}} + v_i v_i^\top \succeq H_{\text{eff}} \succ 0$.

□

6.3 Reducing the Working Set

At a subspace minimizer with a negative multiplier, the index with the most negative multiplier is released from the working set. By Corollary 5.3.2, the multipliers of columns (reduced costs) and of surpluses (duals) are both measured in units of cost, so they can be compared directly.

Theorem 6.3.1 (Update of the effective reduced Hessian, reduction)**.** (iv) Column $r \in \mathcal{W}_x$ is released ($\mathcal{W}_x \leftarrow \mathcal{W}_x \setminus \{r\}$, $\mathcal{N}_x \leftarrow \mathcal{N}_x \cup \{r\}$). The basis gains $z_r = e_r - e_{\text{rep}(r)}$, and with $u = A_{\text{eff}} z_r$,

$$H'_{\text{eff}} = \begin{bmatrix} H_{\text{eff}} & h \\ h^\top & h_{rr} \end{bmatrix}, \qquad h = (A_{\text{eff}} Z_x)^\top u, \qquad h_{rr} = u^\top u. \tag{6.2}$$

(v) The surplus of inequality row i is released ($\mathcal{W}_s \leftarrow \mathcal{W}_s \setminus \{i\}$, $\mathcal{N}_s \leftarrow \mathcal{N}_s \cup \{i\}$). The basis gains e_{n+i}, row i leaves A_{eff}, and

$$H'_{\text{eff}} = H_{\text{eff}} - v_i v_i^\top \tag{6.3}$$

with v_i as in (6.1).

Proof. (iv) $A_{\text{eff}} Z'_x = [A_{\text{eff}} Z_x \quad u]$, and $H'_{\text{eff}} = (A_{\text{eff}} Z'_x)^\top (A_{\text{eff}} Z'_x)$. (v) This is the reverse of case (iii) of Theorem 6.2.4. □

Lemma 6.3.2 (Descent after a reduction)**.** Suppose the reduced Hessian is positive definite after the reduction. Then the next search direction moves the released variable away from its bound: $p_r > 0$ in case (iv), and $(p_s)_i > 0$ in case (v).

Proof. Before the reduction the iterate is a subspace minimizer, so $Z^\top g = 0$. After the reduction, the reduced gradient has a single nonzero entry, the one of the new basis vector, and it equals the multiplier $\nu < 0$ of the released index (Corollary 5.3.2). The new reduced system gives $\eta = -\delta\nu\, \bar{H}'^{-1} e$, where e is the unit vector of the new basis vector. Its own component is $-\delta\nu\, (\bar{H}'^{-1})_{ee} > 0$, because $\bar{H}'^{-1}$ is positive definite. In both cases this component is the change of the released variable ($p_r = \eta_r$ for a column, $(p_s)_i = \eta_i$ for a surplus). □

Unlike an expansion, a reduction can make the reduced Hessian singular: the new column u in case (iv) can lie in the span of the old ones, and the downdate in case (v) can remove the last direction of curvature. This is the subject of Chapter 7.

6.4 Updating the $LDL^\top$ Factorization

The updates of Sections 6.2 and 6.3 are applied to the factorization $H_{\text{eff}}^{\text{emb}} = LDL^\top$, with L unit lower triangular and D diagonal, in the natural order of the indices. We use the sparse direct solver CHOLMOD [3], which provides the three required modifications [5, 6]:

change of the working set	change of $H_{\text{eff}}^{\text{emb}}$	CHOLMOD routine
(i) column leaves $\mathcal{N}_x$	row/column q becomes e_q	`rowdel`
(ii) representative swap	rows/columns of ν_k rewritten	`rowdel` + `rowadd`
(iii) surplus becomes active	$+ v_i v_i^\top$	`updown` (update)
(iv) column enters $\mathcal{N}_x$	row/column r set to $[h; h_{rr}]$	`rowadd`
(v) surplus becomes free	$- v_i v_i^\top$	`updown` (downdate)

6.4.1 Row additions and deletions

Proposition 6.4.1 (Row addition)**.** Let row and column k of $H_{\text{eff}}^{\text{emb}}$ currently be e_k, and partition the indices into $1 = \{1, \dots, k-1\}$, k and $3 = \{k+1, \dots, n_{\max}\}$:

$$L = \begin{bmatrix} L_{11} & & \\ 0 & 1 & \\ L_{31} & 0 & L_{33} \end{bmatrix}, \qquad D = \operatorname{diag}(D_1, 1, D_3).$$

If row and column k are set to (h_1, h_{kk}, h_3), the new factor has

$$\begin{aligned} \ell_{k1} &= D_1^{-1} L_{11}^{-1} h_1 && \text{(triangular solve with the leading block)}, \\ d_k &= h_{kk} - \ell_{k1}^\top D_1 \ell_{k1} && \text{(new pivot)}, \\ \ell_{3k} &= (h_3 - L_{31} D_1 \ell_{k1})/d_k && \text{(new column below } k\text{)}, \\ L'_{33} D'_3 L'^\top_{33} &= L_{33} D_3 L_{33}^\top - d_k\, \ell_{3k} \ell_{3k}^\top && \text{(rank-one downdate)}. \end{aligned}$$

Proof. Multiplying out the new factor, the $(k,1)$ block gives $\ell_{k1}^\top D_1 L_{11}^\top = h_1^\top$, the (k,k) entry gives $\ell_{k1}^\top D_1 \ell_{k1} + d_k = h_{kk}$, the $(3,k)$ block gives $L_{31} D_1 \ell_{k1} + \ell_{3k} d_k = h_3$, and the $(3,3)$ block must reproduce the unchanged $L_{31} D_1 L_{31}^\top + L_{33} D_3 L_{33}^\top$. □

A row deletion is the reverse operation: row and column k are reset to e_k, which requires the rank-one *update* $L_{33} D_3 L_{33}^\top + d_k \ell_{3k} \ell_{3k}^\top$ of the trailing block. Two consequences of Proposition 6.4.1 are important.

- If k is larger than every active index, the trailing block contains no active index, the downdate is trivial, and a row addition costs a single triangular solve with the active part, $O(|\mathcal{N}_x|^2)$ for a dense factor. This is used by the refactorization in Section 6.5.
- The new pivot d_k is the Schur complement of the new entry. If k is the last active index, $d_k = h_{kk} - h^\top H_{\text{eff}}^{-1} h$ with the notation of (6.2), and $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot d_k$. A vanishing pivot therefore signals a singular H'_{eff} (Chapter 7).

6.4.2 Rank-one modifications

The surplus cases (iii) and (v) modify the whole matrix by $\pm v_i v_i^\top$. The factorization $LDL^\top \pm vv^\top = L'D'L'^\top$ is computed by the classical rank-one update and downdate algorithms [10, 5],

which process the columns of L along the path in the elimination tree that starts at the first nonzero of v. Their cost is proportional to the number of entries of L in these columns. A downdate can fail to be positive definite; by the matrix determinant lemma,

$$\det(H_{\text{eff}} - v_i v_i^\top) = \det H_{\text{eff}} \cdot (1 - v_i^\top H_{\text{eff}}^{-1} v_i),$$

so this happens exactly when $\omega_i := 1 - v_i^\top H_{\text{eff}}^{-1} v_i \leq 0$. Chapter 7 shows that for the downdates of the active-set method $\omega_i \geq 0$, and treats the case $\omega_i = 0$.

6.4.3 Sparse solves

The right-hand side of (5.7) vanishes outside $\mathcal{N}_x$. By Lemma 6.1.2, the solve only has to touch the rows of L that can be reached from $\mathcal{N}_x$ in the graph of L, the set $\text{reach}_L(\mathcal{N}_x)$. CHOLMOD supports such solves directly. Their cost is independent of $n_{\max}$, which is what makes a large capacity $n_{\max}$ affordable.

6.4.4 Assembly of a new row

A row addition in case (iv) needs the vector h of (6.2). By (5.12), $h_k = u_k^\top D_\rho u$ with $u = a_r - a_{\text{rep}(r)}$. In an implementation, $D_\rho u$ is scattered into a dense m-vector once, with the entries of the rows of free surpluses set to zero. Then $h_k = a_k^\top (D_\rho u) - a_{\text{rep}(k)}^\top (D_\rho u)$ is computed for every $k \in \mathcal{N}_x$, where the products with the K representatives are computed once and reused. The cost is $O((|\mathcal{N}_x| + K)\,\bar{\nu})$, independent of n. A representative swap (case (ii)) rewrites the rows of ν_k by a row deletion followed by a row addition of the freshly assembled row; this costs the same as applying the formulas of Theorem 6.2.4 and needs no stored copy of H_{eff}.

6.5 Structural Zeros and Periodic Refactorization

A row deletion sets row and column k of L to e_k *numerically*, but a sparse factorization keeps the positions of the removed entries in its sparsity pattern: the pattern of a column of a simplicial factor can only grow. The same holds for rank-one modifications. Over a long sequence of updates, the stored pattern of L therefore becomes the union of the patterns of all factors that ever existed.

Definition 6.5.1 (Stored and effective entries)**.** The **stored entries** of L are the off-diagonal positions in its sparsity pattern. The **effective entries** are the stored entries whose value is nonzero. The difference are the **structural zeros**. The **dense equivalent** is $|\mathcal{N}_x|(|\mathcal{N}_x| - 1)/2$, the number of off-diagonal entries of a dense factor of H_{eff}.

Every solve and every update iterates over the stored entries of the columns it touches, including the structural zeros. Moreover, structural zeros can connect active indices to indices that are no longer active, which enlarges $\text{reach}_L(\mathcal{N}_x)$. The stored pattern only returns to the effective one when the factorization is rebuilt.

Definition 6.5.2 (Refactorization)**.** A **refactorization** resets the factor to the identity and re-activates all indices of $\mathcal{N}_x$ by row additions *in increasing index order*, with the rows assembled as in Section 6.4.

Proposition 6.5.3 (Cost of a refactorization)**.** A refactorization costs $O(|\mathcal{N}_x|^3)$ arithmetic operations for the factor (about $|\mathcal{N}_x|^3/3$ for a dense H_{eff}) and $O(|\mathcal{N}_x|(|\mathcal{N}_x| + K)\,\bar{\nu})$ for the assembly of the rows. Its stored pattern contains no structural zeros.

Proof. When the indices are added in increasing order, every new index is the largest active index, so each row addition is a single triangular solve with the current active part (Proposition 6.4.1), which costs $O(t^2)$ for the t-th index. Summing over $t = 1, \dots, |\mathcal{N}_x|$ gives $O(|\mathcal{N}_x|^3)$, with the constant $1/3$ of a dense Cholesky factorization. Since no row is ever deleted, every stored entry is computed from the current matrix. □

The method refactorizes after every τ updates of the factorization (row additions, row deletions and rank-one modifications), and whenever a pivot has become too small (Chapter 7). The interval τ balances the cost of a rebuild against the growth of the structural zeros, which is roughly proportional to the number of updates. Its effect is studied in Section 8.7.

Remark 6.5.4 (Natural ordering)**.** Sparse direct solvers usually reorder a matrix to reduce fill-in. For H_{eff} this would gain nothing: the effective factor is almost exactly as dense as a dense factor of H_{eff} (Section 8.7), because columns of the same block share their representative and are therefore strongly coupled. The natural ordering has two advantages: index j of the factor is column j of the master problem, and the refactorization of Definition 6.5.2 only appends rows.

6.6 Hot Start Between Column Generation Iterations

Between two master problems, regularized column generation adds columns and moves the stability center (Algorithm 3.1). Both changes can be absorbed without touching the factorization.

Theorem 6.6.1 (Hot start)**.** Let $y = [x; s]$ be the final iterate of a master problem, with its extended ordering and the factorization of H_{eff}.

(i) **New column.** If a column $n + 1 \le n_{\max}$ is added with weight $x_{n+1} = 0$ and placed in the working set $\mathcal{W}_x$, then y remains feasible, $\pi(y)$ is unchanged, and the nullspace basis, A_{eff}, H_{eff} and its factorization are unchanged. The multiplier of the new column is its reduced cost, which is negative because pricing selected it.

(ii) **New center.** If the center changes from $\hat{\pi}$ to $\hat{\pi} + \Delta$, then y remains feasible, the ordering and the factorization are unchanged, and the dual estimate becomes $\pi(y) + \Delta$.

Proof. (i) The convexity constraint of the block of the new column still holds because its weight is zero, and $\bar{A}y$ does not change. The new index is in $\mathcal{W}_x$, so the sets $\mathcal{S}$, $\mathcal{N}_x$, $\mathcal{N}_s$ and with them Z and H_{eff} are unchanged; in the fixed-length representation, index $n + 1$ already carries an identity row. The multiplier is given by Corollary 5.3.2. (ii) The constraints do not involve $\hat{\pi}$, and H_{eff} does not depend on it (Remark 5.2.8). The formula for $\pi(y)$ follows from (3.9). □

After a new column, the previous optimum is a subspace minimizer whose only negative multipliers are those of the new columns. The next master problem therefore typically starts by releasing the new column (case (iv)). After a new center, the reduced gradient changes and the iterate is in general no longer a subspace minimizer; the next master problem starts with a direction computation on the inherited factorization. The surpluses are never added or removed, and their status is carried over like that of the columns.

6.6.1 Center-only iterations

The hot start also explains the behaviour of the center-only iterations of Section 3.4, in which the center moves to the current dual and no column is added. Let Π denote the orthogonal projector onto the range of $\bar{A}Z$.

Proposition 6.6.2 (Center-only iterations)**.** Let y be optimal for the current master problem, with residual $r = b - \bar{A}y$ and dual $\pi = \hat{\pi} + r/\delta$, and let the center move to $\hat{\pi}' = \pi$. Suppose that the next master problem takes one full step and keeps the working set. Then

(i) the new residual is $r' = (I - \Pi)\, r$ and the new dual is $\pi' = \pi + (I - \Pi)\, r/\delta$;

(ii) if the working set stays the same in the following center-only iterations, the primal iterate no longer changes, and every further iteration moves the dual by the fixed vector r'/δ:

$$\pi^{(t+1)} = \pi^{(t)} + \frac{r'}{\delta}, \qquad (\nu_x)_j^{(t+1)} = (\nu_x)_j^{(t)} - \frac{u_j^\top r'}{\delta}, \qquad (\nu_s)_i^{(t+1)} = (\nu_s)_i^{(t)} + \frac{r'_i}{\delta}.$$

Proof. With $\Delta = \pi - \hat{\pi} = r/\delta$, the new dual at the old iterate is $\pi + \Delta$ by Theorem 6.6.1, and the reduced gradient becomes $Z^\top g - (\bar{A}Z)^\top \Delta = -(\bar{A}Z)^\top \Delta$, since $Z^\top g = 0$ at the old optimum. The reduced system (5.5) gives $\eta = \delta\, \bar{H}^{-1}(\bar{A}Z)^\top \Delta$, hence $\bar{A}p = \bar{A}Z\eta = \delta\, \Pi\Delta$. After the full step, $\pi' = \pi + \Delta - \frac{1}{\delta}\bar{A}p = \pi + (I - \Pi)\Delta$, and $r' = \delta(\pi' - \hat{\pi}') = (I - \Pi)\, r$. This is (i).

For (ii), r' is orthogonal to the range of $\bar{A}Z$. In the next center-only iteration, $\Delta' = r'/\delta$, so the reduced gradient $-(\bar{A}Z)^\top \Delta'$ vanishes, the direction is zero, and y does not move. The dual becomes $\pi' + \Delta'$, the residual stays r', and the same argument applies again. The multipliers change by (4.2): $g_j - g_{\mathrm{rep}(j)}$ changes by $-(a_j - a_{\mathrm{rep}(j)})^\top \Delta'$, and $g_{n+i} = \pi_i$ by Δ'_i. □

Proposition 6.6.2 shows that in a center-only phase the dual walks along a fixed direction with step length $\|r'\|/\delta$, while the primal iterate stays put. The walk ends when a multiplier changes sign, so that the working set changes, or when pricing finds an improving column. If the residual r' is small, many such iterations are needed, which explains the long tail of center-only iterations observed in Section 8.4. Each of them is cheap: one direction computation, which returns zero, and one multiplier test.

6.7 The Complete Method

Algorithm 6.1 combines the results of Chapters 4 to 6. It is called in the first line of the loop of regularized column generation (Algorithm 3.1), and its state, consisting of the iterate, the extended ordering and the factorization, is kept from one call to the next.

Algorithm 6.1: Hot-Started Nullspace Active-Set Method

1: **Input:** state of the previous call (iterate y, ordering, factor of H_{eff}), new columns, new center $\hat{\pi}$
2: Put the new columns into $\mathcal{W}_x$ with weight 0 ▷ Theorem 6.6.1
3: $\pi \leftarrow \hat{\pi} + (b - \bar{A}y)/\delta$
4: **repeat**
5: **if** the last step was not a full step **then**
6: Compute p by Algorithm 5.1 (lines 2–4)
7: **else**
8: $p \leftarrow 0$ ▷ Proposition 5.2.6
9: **end if**
10: **if** $p \approx 0$ **then**
11: Compute the multipliers (Corollary 5.3.2)
12: **if** all multipliers $\geq -\varepsilon$ **then return** y, π, μ
13: **end if**
14: Release the most negative index: `rowadd` (column) or downdate (surplus)
15: If the new pivot is too small, resolve the singularity (Algorithm 7.1)
16: **else**
17: Ratio test over $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$; $y \leftarrow y + \alpha p$; $\pi \leftarrow \pi - \frac{\alpha}{\delta}\bar{A}p$
18: **if** blocked by q **then**
19: Expand: `rowdel` ($q \in \mathcal{N}_x$), representative swap ($q \in \mathcal{S}$), or update ($q \in \mathcal{N}_s$)
20: **end if**
21: **end if**
22: Refactorize if τ updates have passed since the last refactorization
23: **until** returned

Chapter 7

Singular Reduced Hessians

The nullspace method requires the reduced Hessian to be positive definite (Assumption 4.3.3 (ii)). The Hessian of the regularized master problem is only positive semidefinite, and its rank is bounded by the number of linking rows, while the number of columns keeps growing. It is therefore unavoidable that a reduction of the working set occasionally produces a nullspace direction along which the objective has no curvature.

This chapter shows where such singularities can arise, how they are detected from the pivots of the factorization, and how they are resolved. The resolution is a **zero-curvature step**: along a direction in the kernel of the reduced Hessian, the objective decreases linearly, and the step is taken until a variable reaches its bound. We prove that adding this variable to the working set removes the singularity, so that the interrupted reduction can be completed. The treatment covers released columns and released surpluses with one argument, and keeps every working-set variable exactly at its bound.

7.1 Where Singularity Can Arise

By Corollary 5.2.5, the reduced Hessian is positive definite if and only if $H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x)$ is, i.e. if and only if the columns $A_{\text{eff}} z_j$, $j \in \mathcal{N}_x$, are linearly independent. Proposition 6.2.5 shows that expansions preserve this property. Singularity can therefore only arise in a reduction.

Proposition 7.1.1 (Singular reductions)**.** Let H_{eff} be positive definite.

(iv) After releasing column r, H'_{eff} of (6.2) is singular if and only if $A_{\text{eff}} z_r \in \operatorname{span}\{A_{\text{eff}} z_k : k \in \mathcal{N}_x\}$, i.e. if and only if the Schur complement $\sigma_r = h_{rr} - h^\top H_{\text{eff}}^{-1} h$ vanishes.

(v) After releasing the surplus of row i, $H'_{\text{eff}} = H_{\text{eff}} - v_i v_i^\top$ is singular if and only if $\omega_i = 1 - v_i^\top H_{\text{eff}}^{-1} v_i = 0$.

Moreover $\sigma_r \geq 0$ and $0 \leq \omega_i \leq 1$.

Proof. (iv) H'_{eff} is the Gram matrix of the columns of $[A_{\text{eff}} Z_x \;\; A_{\text{eff}} z_r]$, and $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot \sigma_r$. (v) By the matrix determinant lemma, $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot \omega_i$. In both cases H'_{eff} is a Gram matrix, hence positive semidefinite, so $\det H'_{\text{eff}} \geq 0$ and therefore $\sigma_r \geq 0$ and $\omega_i \geq 0$. Finally $\omega_i \leq 1$ because $v_i^\top H_{\text{eff}}^{-1} v_i \geq 0$. □

Proposition 7.1.1 also shows that a downdate in case (v) can never make H_{eff} indefinite: in exact arithmetic, the worst case is a singular matrix.

7.2 Detection

Testing σ_r or ω_i before the update would cost an additional solve for every reduction. Instead, the update is performed, and the pivots of the factorization are inspected afterwards.

Definition 7.2.1 (Pivot invariant)**.** For $k \in \mathcal{N}_x$, let $\gamma_k = \|a_k - a_{\text{rep}(k)}\|^2$ be the **pivot scale** of index k (with all rows, not only those of A_{eff}). The factorization satisfies the **pivot invariant** with tolerance $\varepsilon_{\text{piv}} > 0$ if

$$D_{kk} \;>\; \varepsilon_{\text{piv}}\, \gamma_k \qquad \text{for all } k \in \mathcal{N}_x.$$

The scale γ_k uses all rows on purpose: a column whose coefficients lie entirely in rows with free surpluses has $(H_{\text{eff}})_{kk} = 0$, and a scale based on H_{eff} would hide this singularity. The determinant of H'_{eff} is the product of the pivots. If all pivots satisfied the invariant before the update, a vanishing σ_r or ω_i must show up as a pivot that violates it after the update. This detection is a numerical heuristic rather than a theorem about floating-point arithmetic, but it follows directly from the determinant identities of Proposition 7.1.1. When it fires, the update is undone by a refactorization with the previous working set (Definition 6.5.2), which also removes any large entries created by dividing by a tiny pivot. We use $\varepsilon_{\text{piv}} = 10^{-10}$.

7.3 Zero-Curvature Directions

Assume now that the iterate y is a subspace minimizer for the working set $\mathcal{W}$, that the index to be released has a negative multiplier, and that its release would make the reduced Hessian singular. We construct a direction along which the objective decreases linearly.

7.3.1 Released column

Theorem 7.3.1 (Zero-curvature direction, column)**.** Let $r \in \mathcal{W}_x$ have the multiplier $(\nu_x)_r < 0$, and suppose $A_{\text{eff}} z_r = \sum_{k \in \mathcal{N}_x} t_k A_{\text{eff}} z_k$, i.e. $t = H_{\text{eff}}^{-1} h$ with h from (6.2). Define $d = [d_x; d_s]$ by

$$d_x = z_r - \sum_{k \in \mathcal{N}_x} t_k z_k, \qquad (d_s)_\ell = (A_I d_x)_\ell \ \ (\ell \in \mathcal{N}_s), \qquad (d_s)_\ell = 0 \ \ (\ell \in \mathcal{W}_s), \tag{7.1}$$

where z_k denotes the column part $e_k - e_{\text{rep}(k)} \in \mathbb{R}^n$. Then

(i) $C d_x = 0$, and $d_\ell = 0$ for every $\ell \in \mathcal{W} \setminus \{r\}$;

(ii) $\bar{A} d = 0$;

(iii) $\nabla f(y)^\top d = (\nu_x)_r < 0$.

Proof. (i) Every z_k has block sums zero and is supported on $\{k, \text{rep}(k)\}$, so $C d_x = 0$, and d_x vanishes on $\mathcal{W}_x \setminus \{r\}$. The surplus part vanishes on $\mathcal{W}_s$ by definition.
(ii) Consider the rows of $\bar{A} d = [A_E d_x;\ A_I d_x - d_s]$. On the rows of A_{eff} (the equality rows and the inequality rows with active surplus), $(\bar{A} d)_\ell = (A_{\text{eff}} d_x)_\ell$, and $A_{\text{eff}} d_x = A_{\text{eff}} z_r - \sum_k t_k A_{\text{eff}} z_k = 0$ by assumption. On an inequality row $\ell \in \mathcal{N}_s$, $(\bar{A} d)_\ell = (A_I d_x)_\ell - (d_s)_\ell = 0$ by construction.
(iii) At a subspace minimizer the reduced gradient vanishes, so $g^\top z_k = 0$ for $k \in \mathcal{N}_x$ and $\pi_\ell = 0$ for $\ell \in \mathcal{N}_s$ (Corollary 5.3.2). Hence

$$g^\top d = g_x^\top z_r - \sum_k t_k \, g_x^\top z_k + \sum_{\ell \in \mathcal{N}_s} \pi_\ell (d_s)_\ell = g_r - g_{\text{rep}(r)} = (\nu_x)_r,$$

where the last equality is Corollary 5.3.2. □

7.3.2 Released surplus

Theorem 7.3.2 (Zero-curvature direction, surplus)**.** Let the surplus of row $i \in \mathcal{W}_s$ have the multiplier $(\nu_s)_i = \pi_i < 0$, and suppose $\omega_i = 0$. Let $t = H_{\text{eff}}^{-1} v_i$, so that $v_i^\top t = 1$. Define $d = [d_x; d_s]$ by

$$d_x = \sum_{k \in \mathcal{N}_x} t_k z_k, \qquad (d_s)_\ell = (A_I d_x)_\ell \ \ (\ell \in \mathcal{N}_s \cup \{i\}), \qquad (d_s)_\ell = 0 \ \text{ otherwise.} \tag{7.2}$$

Then

(i) $C d_x = 0$, d_x vanishes on $\mathcal{W}_x$, and $(d_s)_i = 1$;

(ii) $\bar{A} d = 0$;

(iii) $\nabla f(y)^\top d = \pi_i < 0$.

Proof. (i) The first two claims are as in Theorem 7.3.1. Further, $(A_I d_x)_i = (A_I)_{i,:} Z_x t = v_i^\top t = 1$.
(ii) Let A'_{eff} be A_{eff} without row i. Then $(A'_{\text{eff}} Z_x)^\top (A'_{\text{eff}} Z_x)\, t = (H_{\text{eff}} - v_i v_i^\top)\, t = v_i - v_i (v_i^\top t) = 0$, so $A'_{\text{eff}} Z_x t = A'_{\text{eff}} d_x = 0$. This covers all rows of $\bar{A} d$ except the inequality rows in $\mathcal{N}_s \cup \{i\}$, on which $(\bar{A} d)_\ell = (A_I d_x)_\ell - (d_s)_\ell = 0$ by construction.
(iii) As before, $g^\top z_k = 0$ for $k \in \mathcal{N}_x$ and $\pi_\ell = 0$ for $\ell \in \mathcal{N}_s$, so $g^\top d = \pi_i \, (d_s)_i = \pi_i$. □

Remark 7.3.3. For a "$\le$" row with slack coefficient $\kappa_i = +1$ (Remark 5.3.3), the column part is scaled by $-\kappa_i$, $d_x = -\kappa_i \sum_k t_k z_k$, and the surplus part is $(d_s)_\ell = -\kappa_\ell (A_I d_x)_\ell$. Then again $(d_s)_i = 1$ and $g^\top d = -\kappa_i \pi_i = (\nu_s)_i < 0$.

Corollary 7.3.4 (Linear decrease)**.** For the directions of Theorems 7.3.1 and 7.3.2,

$$f(y+\alpha d) = f(y) + \alpha\,\nabla f(y)^\top d, \qquad \pi(y+\alpha d) = \pi(y) \qquad \text{for all } \alpha.$$

Proof. The quadratic term of $f(y+\alpha d)$ is $\frac{\alpha^2}{2\delta}\|\bar{A}d\|^2 = 0$, and π depends on y only through $\bar{A}y$. □

Remark 7.3.5 (Fully masked columns)**.** If all nonzero coefficients of $u = a_r - a_{\mathrm{rep}(r)}$ lie in rows with free surpluses, then $A_{\mathrm{eff}}z_r = 0$ and $h = 0$, $h_{rr} = 0$. Theorem 7.3.1 then applies with $t = 0$: the direction is z_r together with the surplus components that absorb $A_I z_r$.

7.4 The Zero-Curvature Step Removes the Singularity

Along d the objective decreases linearly, so the method moves as far as feasibility allows.

Lemma 7.4.1 (Finite step)**.** The directions of Theorems 7.3.1 and 7.3.2 have at least one negative component. Hence the ratio test

$$\alpha^\star = \min_{\ell:\, d_\ell<0} -\frac{y_\ell}{d_\ell}$$

is finite, and it is attained at a **blocking index** q in $\mathcal{S}\cup\mathcal{N}_x\cup\mathcal{N}_s$.

Proof. Suppose $d \geq 0$. Every block sum of d_x is zero (Theorems 7.3.1 and 7.3.2 (i)), so $d_x \geq 0$ forces $d_x = 0$. Then $\bar{A}d = [0; -d_s] = 0$ forces $d_s = 0$, and $d = 0$ contradicts $g^\top d < 0$. The blocking index cannot lie in $\mathcal{W}\setminus\{r\}$ since d vanishes there, and it is not the released index, whose component is positive ($d_r = 1$ or $(d_s)_i = 1$). □

After the step, the blocking index q is added to the working set by an expansion (Section 6.2), and the release is attempted again. The following theorem shows that this attempt cannot fail for the same reason.

Theorem 7.4.2 (Removal of the dependency)**.** Let $V = \mathrm{Null}(D_{\mathcal{W}\setminus\{\rho\}})$, where ρ is the released index (r or the surplus i), and let $V' = V \cap \{y_q = 0\}$. Then the reduced Hessian of the working set $(\mathcal{W}\setminus\{\rho\})\cup\{q\}$ is positive definite. In other words, the retried release succeeds.

Proof. The reduced Hessian of a working set is positive definite if and only if $\bar{A}$ is injective on the corresponding nullspace (Corollary 5.2.5). Before the release, $\bar{A}$ was injective on $\mathrm{Null}(D_{\mathcal{W}})$, and V has exactly one more dimension than $\mathrm{Null}(D_{\mathcal{W}})$. Hence $\ker\bar{A}\cap V$ has dimension at most one, and it contains $d \neq 0$ by Theorems 7.3.1 and 7.3.2 (ii), so $\ker\bar{A}\cap V = \mathrm{span}\{d\}$. Since $d_q \neq 0$, $d \notin V'$, and therefore $\ker\bar{A}\cap V' \subseteq \mathrm{span}\{d\}\cap V' = \{0\}$.

□

Proposition 7.4.3 (The iterate stays a subspace minimizer)**.** After the zero-curvature step and the expansion by q, the iterate is a subspace minimizer of the new working set, and the multiplier of the released index is unchanged.

Proof. By Corollary 7.3.4, π and hence the gradient do not change. Expanding the working set removes a nullspace vector or changes the representative of a block; in the latter case the new reduced gradient entries are differences $g_j - g_{r'} = (g_j - g_q) - (g_{r'} - g_q) = 0$ of entries that were zero. The multiplier of ρ is computed from the gradient, which has not changed. □

Remark 7.4.4 (The representative of the released column blocks)**.** In Theorem 7.3.1, the blocking index may be the representative $\mathrm{rep}(r)$ of the block of r. Then r, which now has the positive weight $\alpha^\star$, can simply take over as the new representative (case (ii) of Lemma 6.2.1 with replacement r). This completes the release at once.

Algorithm 7.1 summarizes the procedure. Each pass either completes the release or adds one more index to the working set. The number of passes is therefore bounded by the number of free variables.

Algorithm 7.1: Release with Singularity Resolution

Input: subspace minimizer y, index ρ (column r or surplus i) with negative multiplier
loop
  Apply the reduction update (`rowadd` for a column, downdate for a surplus)
  if the pivot invariant holds **then**
    return ▷ release completed
  end if
  Undo by refactorization with ρ still in the working set
  Solve $H_{\mathrm{eff}}t = h$ (column) or $H_{\mathrm{eff}}t = v_i$ (surplus)
  Form d by (7.1) or (7.2)
  Ratio test along d; $y \leftarrow y + \alpha^\star d$ ▷ π is unchanged
  if $\rho = r$ and $q = \mathrm{rep}(r)$ **then**
    Make r the representative, put q into $\mathcal{W}_x$; **return** ▷ Remark 7.4.4
  end if
  Expand the working set by q ▷ Theorem 7.4.2: the next pass succeeds
end loop

Remark 7.4.5 (Why not remove the largest coefficient?)**.** A simpler rule moves the index $k = \arg\max_{k \in \mathcal{N}_x} |t_k|$ of the largest coefficient of the linear dependence into the working set and retries. This also removes the dependency, but it ignores the value of x_k. If $x_k > 0$, the working-set constraint $x_k = 0$ is violated by the iterate, and its multiplier no longer certifies optimality: the method can stop with $x_k > 0$ and a positive multiplier. The zero-curvature step avoids this. The iterate stays feasible, every working-set variable is exactly at its bound, the objective decreases strictly, and the index added to the working set is the one selected by a ratio test rather than by a heuristic. The same argument covers released columns and released surpluses.

Chapter 8

Computational Results

This chapter evaluates the hot-started nullspace active-set method on generated instances of the **airline crew scheduling problem**. Every instance is solved in two variants: with equality coverage rows only, and with 30% of the coverage rows relaxed to "$\geq$" inequalities.

We first check **correctness**: every column generation method must reproduce the optimum of the full linear program, which is still small enough to be solved directly. We then study the **convergence behaviour** of regularized column generation, which consists of two clearly separated kinds of iterations, and the **computational time**, broken down by solver, by iteration and by operation inside the active-set method.

The last two sections look inside the method. We follow the sizes of the nullspace sets over a run with inequality rows, and we measure the **contents of the factorization**: how many of the stored entries are actually nonzero, how fast structural zeros accumulate, and how much periodic refactorization is worth.

8.1 Test Problem: Airline Crew Scheduling

8.1.1 The model

In the crew scheduling problem, K pilots have to be assigned schedules that together operate a set of m flights (Example 2.1.4). The schedules of a pilot are the source–sink paths in a **time–space network**: a directed acyclic graph $G = (V, E)$ whose nodes represent states of a pilot at given times, and whose edges represent the allowed transitions between them. Edge e costs pilot k the amount $c_{e,k}$, and it operates a subset of the flights, recorded in the **task matrix** $A_{\text{task}} \in \{0,1\}^{m \times |E|}$. With $x_k \in \{0,1\}^{|E|}$ the edge vector of the path of pilot k and F the node–edge incidence matrix of G, the linear relaxation reads

$$\min_{x_1,\dots,x_K} \quad \sum_{k=1}^{K} c_k^{\top} x_k \tag{8.1a}$$

$$\text{s.t.} \quad \Big(\sum_{k=1}^{K} A_{\text{task}}\, x_k\Big)_i = 1 \qquad (i \in \mathcal{E}), \tag{8.1b}$$

$$\Big(\sum_{k=1}^{K} A_{\text{task}}\, x_k\Big)_i \geq 1 \qquad (i \in \mathcal{I}), \tag{8.1c}$$

$$F x_k = d, \quad 0 \leq x_k \leq 1 \qquad (k = 1, \dots, K), \tag{8.1d}$$

where d has the entry -1 at the source, $+1$ at the sink and 0 elsewhere. This is a multi-block linear program (Definition 2.1.1) with identical block polytopes. A coverage row in $\mathcal{E}$ requires a flight to be operated exactly once; a row in $\mathcal{I}$ requires it at least once, which allows a pilot to travel on a flight as a passenger. The columns of the master problem are complete schedules, and the pricing problem (2.5) of pilot k is a shortest path problem with edge weights $c_{e,k} - (A_{\text{task}}^{\top}\pi)_e$. These weights can be negative, but the graph is acyclic, so one pass of dynamic programming in topological order solves it in $O(|V| + |E|)$.

8.1.2 Instance generator

The instances are generated randomly but reproducibly.

- **Network.** The nodes $1, \dots, |V|$ are ordered in time. A backbone edge $v \to v+1$ connects consecutive nodes, which guarantees a source–sink path. Every skip edge $v \to w$ with $w \geq v + 2$ exists independently with a fixed probability. An edge costs $(w - v) + U(0,1)$, multiplied by $1 + 0.1\,U(0,1)$ for each pilot separately.
- **Flights.** Each skip edge operates a random set of up to ten flights whose position in time is close to that of the edge. A flight is only attached to edges that overlap in time, so that a single schedule operates every flight at most once. The task matrix therefore has a banded structure.
- **Feasibility.** Only flights that can be covered exactly once in the linear relaxation are kept, so that the equality variant is feasible.
- **Inequality rows.** In the inequality variant, a random 30% of the coverage rows are turned into "≥ 1" rows.

Table 8.1 lists the four instance sizes. The last column is the number of variables of the full edge-based formulation (8.1), which HiGHS solves directly to provide the reference optimum $z^{\star}$.

instance	$\|V\|$	$\|E\|$	K	m	m_I (inequality variant)	variables of (8.1)
I1	50	156	4	38	11	624
I2	150	1 146	7	225	68	8 022
I3	175	1 546	10	356	107	15 460
I4	225	2 434	13	480	144	31 642

Table 8.1: Test instances. K is the number of pilots, m the number of coverage rows, and m_I the number of "$\geq$" rows in the inequality variant.

8.2 Implementation and Setup

All methods are implemented in Julia 1.12 [1] in one framework: the column generation driver, the pricing algorithm, the column pool and the statistics are shared, and only the solver of the restricted master problem differs. We compare five methods:

method	master problem
Exact LP (HiGHS)	none: formulation (8.1) solved directly by the HiGHS simplex method
LP CG (HiGHS)	classical linear master problem, HiGHS simplex, warm-started
Explicit QP CG (HiGHS)	explicit regularized master problem (3.3), HiGHS QP solver
Eliminated QP CG (HiGHS)	eliminated regularized master problem (3.5), HiGHS QP
Hot-started QPAS CG	eliminated regularized master problem, Algorithm 6.1

HiGHS [11] (version 1.15) is used through a persistent model, to which new columns are added incrementally, so that its LP solves are warm-started. The eliminated QP is only defined for equality rows in HiGHS form, since it requires the elimination of w without surplus variables; it is therefore run on the equality variant only. The active-set method uses CHOLMOD 5.3 [3] for the factorization and its updates.

All column generation runs use the same settings: the initial columns are the cheapest path of every pilot, pricing visits the pilots in random order and adds the first improving column, the proximal point rule moves the center after every iteration, and $\delta = 1$, $\tau = 10^{-6}$, $\varepsilon_{\mathrm{rc}} = 10^{-6}$. The active-set method uses the column capacity $n_{\max} = 20\,000$ and the refactorization interval 150 unless stated otherwise. The HiGHS QP masters are stopped after 600 seconds and the LP master after 900 seconds. All experiments ran single-threaded on an AMD Ryzen 7 7700 processor with 32 GB of memory under Windows 11. Times of the active-set method are the faster of two runs.

8.3 Correctness

Tables 8.2 and 8.3 compare all methods on both variants. Besides the status, the number of column generation iterations, the number of generated columns and the times, they report three accuracy measures relative to the reference optimum $z^\star$:

- the relative error of the cost, $(c^\top \lambda - z^\star)/|z^\star|$;
- the relative gap of the best Lagrangian bound, $(z^\star - \theta)/|z^\star|$, which is nonnegative for every valid bound (Proposition 2.3.4);

- the infeasibility $\|b - A\lambda\|_\infty$ of the final weights in the linking rows (for inequality rows only the violated part).

All methods reach the optimum. Every column generation run that finishes within its time limit stops with an optimal status, and the Lagrangian bound agrees with $z^\star$ to within 10^{-6} relative or better. This confirms the exact termination criterion of Theorem 3.4.1 in practice, including the sign conditions of the inequality rows: the bound would be $-\infty$ if the projected duals violated them (Remark 2.3.5).

The accuracy of the stabilized methods. The stabilized methods stop when $\|\pi - \hat{\pi}\|_\infty \leq \tau \max(1, \|\pi\|_\infty)$, which by Proposition 3.4.2 allows an infeasibility of up to $\delta\tau \max(1, \|\pi\|_\infty)$ in the linking rows. The infeasibility column shows exactly this order of magnitude, about 10^{-5} for the larger instances, where the duals are of order 10 to 100. The cost $c^\top \lambda$ of such a slightly infeasible solution can be off by more than the infeasibility itself: on I2 it is $2 \cdot 10^{-5}$ below $z^\star$. The Lagrangian bound, which certifies the optimal value, is unaffected and agrees with $z^\star$ to about 10^{-7}. A tighter τ removes the cost error at the price of a few more center-only iterations. In a test on a smaller instance with $\delta = 5$, the cost error dropped from 10^{-6} for $\tau = 10^{-6}$ to 10^{-15} for $\tau = 10^{-9}$, with one additional iteration.

The active-set method and HiGHS solve the same problem. On instance I1, where all stabilized methods finish, the explicit and eliminated HiGHS QPs and the active-set method perform almost the same number of column generation iterations and generate the same number of columns: they follow the same sequence of master problems, as they should. The small differences come from the accuracy of the master solves, which affects the duals passed to pricing and hence the order in which columns are found.

	method	status	CG iter.	columns	time [s]	master [ms/iter.]	cost error	bound gap	infeasibility
I1	Exact LP (HiGHS)	opt.	–	–	0.01	–	0	0	0
	LP CG (HiGHS)	opt.	204	207	0.06	0.299	0	0	$4.4 \cdot 10^{-16}$
	Explicit QP CG (HiGHS)	opt.	3 960	130	8.22	2.062	$5.4 \cdot 10^{-10}$	$3.5 \cdot 10^{-10}$	$2.9 \cdot 10^{-9}$
	Eliminated QP CG (HiGHS)	opt.	3 960	130	5.74	1.441	$5.4 \cdot 10^{-10}$	$3.5 \cdot 10^{-10}$	$2.9 \cdot 10^{-9}$
	Hot-started QPAS CG	opt.	3 928	129	0.05	0.010	0	0	$2.4 \cdot 10^{-14}$
I2	Exact LP (HiGHS)	opt.	–	–	1.56	–	0	0	0
	LP CG (HiGHS)	opt.	2 410	2 416	73.60	30.469	$-6.2 \cdot 10^{-13}$	$9.1 \cdot 10^{-11}$	$1.3 \cdot 10^{-12}$
	Explicit QP CG (HiGHS)	time	6 249	1 156	600.04	95.924	$-2.5 \cdot 10^{-4}$	$3.9 \cdot 10^{-6}$	$6.7 \cdot 10^{-4}$
	Eliminated QP CG (HiGHS)	time	5 629	1 149	600.07	106.439	$-5.1 \cdot 10^{-4}$	$8.2 \cdot 10^{-6}$	$1.4 \cdot 10^{-3}$
	Hot-started QPAS CG	opt.	11 974	1 268	2.67	0.200	$-2.2 \cdot 10^{-5}$	$1.7 \cdot 10^{-7}$	$5.8 \cdot 10^{-5}$
I3	Exact LP (HiGHS)	opt.	–	–	6.49	–	0	0	0
	LP CG (HiGHS)	opt.	4 679	4 688	649.50	138.712	0	$1.4 \cdot 10^{-12}$	$7.7 \cdot 10^{-13}$
	Hot-started QPAS CG	opt.	8 961	1 651	7.70	0.825	$8.7 \cdot 10^{-9}$	$5.4 \cdot 10^{-9}$	$1.4 \cdot 10^{-5}$
I4	Exact LP (HiGHS)	opt.	–	–	23.96	–	0	0	0
	Hot-started QPAS CG	opt.	3 216	2 408	22.30	6.865	$-2.7 \cdot 10^{-8}$	$1.6 \cdot 10^{-10}$	$7.8 \cdot 10^{-6}$

Table 8.2: Equality variant: all methods. “time” denotes a method stopped by its time limit. Master times are averages per column generation iteration.

	method	status	CG iter.	columns	time [s]	master [ms/iter.]	cost error	bound gap	infeasibility
I1	Exact LP (HiGHS)	opt.	–	–	0.01	–	0	0	0
	LP CG (HiGHS)	opt.	138	141	0.03	0.220	0	0	$1.8 \cdot 10^{-15}$
	Explicit QP CG (HiGHS)	opt.	370	89	0.56	1.489	$5.1 \cdot 10^{-11}$	$2.1 \cdot 10^{-10}$	$1.2 \cdot 10^{-9}$
	Hot-started QPAS CG	opt.	369	94	0.01	0.020	0	0	$8.4 \cdot 10^{-15}$
I2	Exact LP (HiGHS)	opt.	–	–	1.32	–	0	0	0
	LP CG (HiGHS)	opt.	1 860	1 866	30.51	16.344	$1.1 \cdot 10^{-12}$	$1.1 \cdot 10^{-10}$	$1.0 \cdot 10^{-11}$
	Explicit QP CG (HiGHS)	opt.	1 573	715	95.51	60.638	$-8.3 \cdot 10^{-7}$	$5.5 \cdot 10^{-11}$	$1.5 \cdot 10^{-5}$
	Hot-started QPAS CG	opt.	1 630	716	0.78	0.457	$-1.9 \cdot 10^{-7}$	0	$5.6 \cdot 10^{-6}$
I3	Exact LP (HiGHS)	opt.	–	–	5.43	–	0	0	0
	LP CG (HiGHS)	opt.	3 761	3 770	302.25	80.270	0	$3.2 \cdot 10^{-13}$	$3.6 \cdot 10^{-13}$
	Hot-started QPAS CG	opt.	5 460	1 377	5.13	0.906	$-4.7 \cdot 10^{-9}$	$2.7 \cdot 10^{-10}$	$4.0 \cdot 10^{-6}$
I4	Exact LP (HiGHS)	opt.	–	–	21.95	–	0	0	0
	Hot-started QPAS CG	opt.	2 856	2 099	17.38	6.018	$5.9 \cdot 10^{-10}$	$4.8 \cdot 10^{-11}$	$4.3 \cdot 10^{-6}$

Table 8.3: Inequality variant (30% “$\geq$” rows). The eliminated HiGHS QP master does not support inequality rows.

8.4 Convergence and the Two Phases of Regularized Column Generation

Figure 8.1 shows the convergence of the master objective on instance I2 for all column generation methods, together with the change of the duals from one iteration to the next.

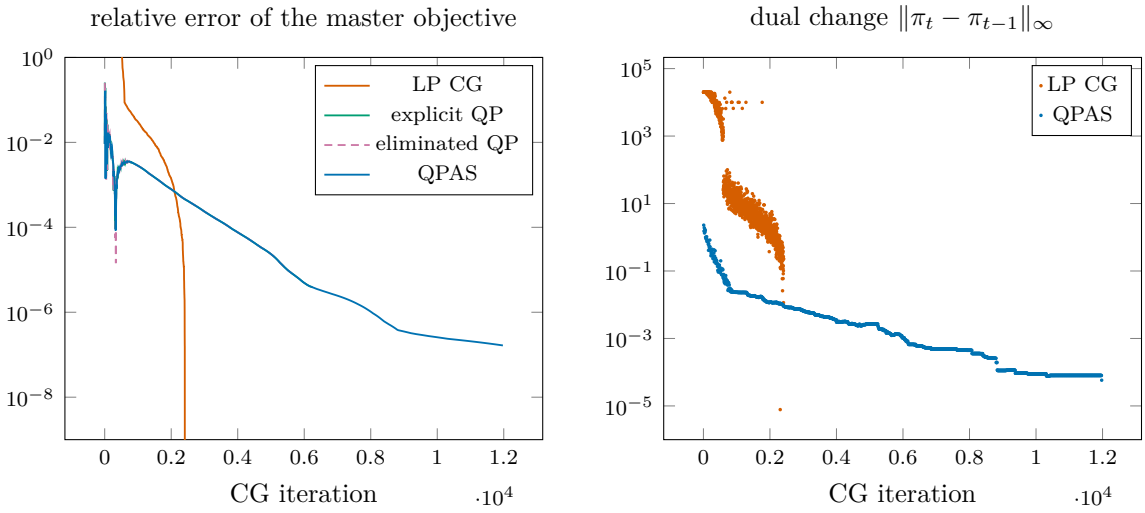


Figure 8.1: Instance I2 (equality variant). Left: relative distance of the master objective to $z^\star$. Right: change of the duals between consecutive iterations. The two HiGHS QP masters stop at their time limit after about 6 000 iterations.

Stabilization. The duals of the linear master problem jump by up to $2 \cdot 10^4$ in the first iterations, when the artificial Big-M variables are still active, and by about 2 in a typical iteration of the second half of the run. The regularized duals change by about 10^{-4} in a typical iteration of the second half. The linear master problem, however, reaches the optimum in fewer iterations on this instance. Its jumps explore the dual space quickly, while the proximal term keeps the regularized duals close to the center, so that they need many small steps. Regularization therefore does not reduce the number of iterations here. What it buys is iterations whose master problems differ only slightly from each other, which is exactly what the hot start exploits: an iteration of the active-set method costs 0.2 ms on average, against 30 ms for the warm-started LP master (Table 8.2).

The QP masters follow the same path. The curves of the explicit QP, the eliminated QP and the active-set method are almost indistinguishable over the first 6 000 iterations, after which the HiGHS QP masters run out of time.

The right panel of Fig. 8.1 shows a second feature of the regularized methods: from some point on, the duals change by very small amounts in most iterations. These are the center-only iterations of Section 3.4. Figure 8.2 shows how the two kinds of iterations are distributed over a run.

Two phases. A run starts with a **column phase**, in which (almost) every iteration adds a column. It is followed by a **proximal phase**, in which most iterations are center-only iterations that move the center towards the restricted dual optimum, interrupted by occasional iterations that find a new column because the duals have moved. The phases are not strictly separated: columns keep appearing until close to the end. Table 8.4 quantifies this for all instances.

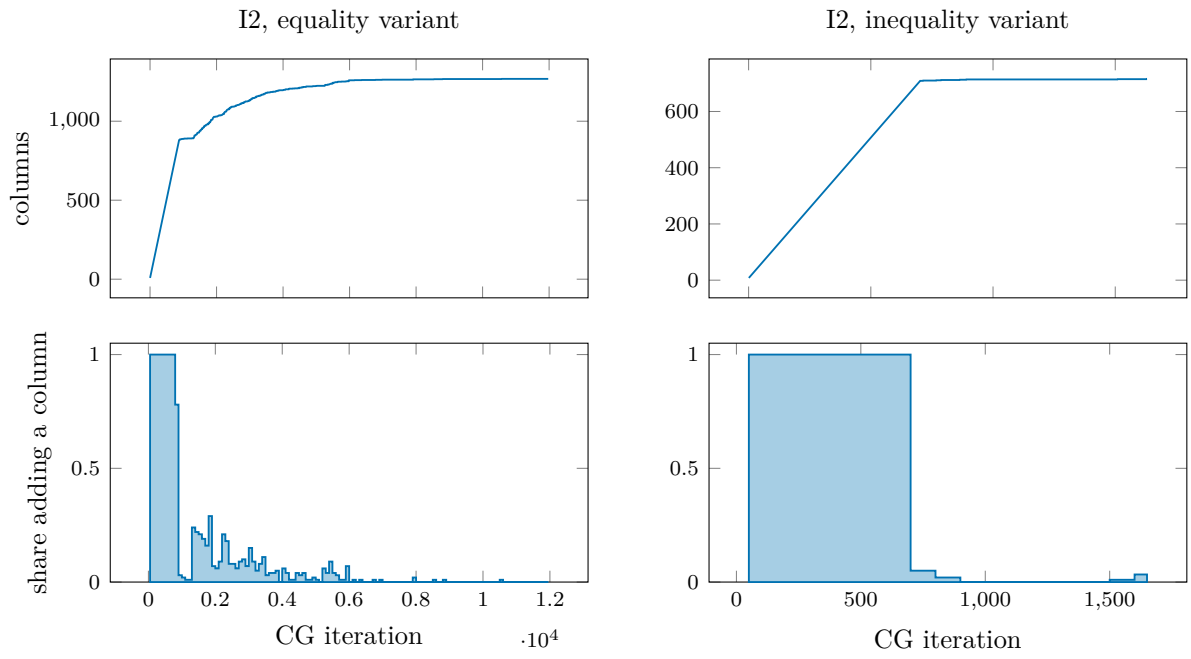


Figure 8.2: The two phases of regularized column generation (active-set method, instance I2). Top: number of columns. Bottom: share of the iterations in each window of 100 iterations that add a column; the remaining iterations are center-only iterations.

The two kinds of iterations differ sharply in cost. A column-adding iteration needs on average tens of active-set iterations, since the new column is released and the working set adapts to it. A center-only iteration needs at most two on average: as shown in Proposition 6.6.2, once the working set has settled, the direction is zero, and the master problem is solved by a single multiplier test. The length of the proximal phase varies a lot between instances. On I2 and I3 (equality variant) center-only iterations are the large majority, on I4 they are a minority. In all cases their share of the master time is much smaller than their share of the iterations.

Figure 8.3 shows how the optimality measures evolve over the two phases for the active-set method on I2. In the column phase, the cost error, the error of the master objective and the infeasibility decrease together. In the proximal phase, the Lagrangian bound becomes available, since all pilots are priced in every center-only iteration. The bound approaches $z^\star$ from below at the same rate at which the master objective approaches it, so the two errors are indistinguishable in the plot. The infeasibility, and with it the cost error, decrease more slowly and in steps: each step is a change of the working set that ends one of the dual walks of Proposition 6.6.2.

instance	ineq.	CG iter.	with column	center-only	first center-only	last column	mean inner iter. column	mean inner iter. center-only	master time center-only
I1	–	3 928	125	3 803	109	3 926	10.7	1.0	82%
I1	30%	369	90	279	82	367	12.8	1.4	29%
I2	–	11 974	1 261	10 713	876	10 571	34.2	1.8	48%
I2	30%	1 630	709	921	703	1 629	45.6	1.5	8%
I3	–	8 961	1 641	7 320	1 616	6 166	65.7	1.7	16%
I3	30%	5 460	1 367	4 093	1 351	5 459	66.3	1.7	13%
I4	–	3 216	2 395	821	2 389	2 568	87.2	2.0	1%
I4	30%	2 856	2 086	770	2 082	2 854	92.7	1.8	1%

Table 8.4: Column-adding and center-only iterations of the active-set method. “inner iter.” counts the iterations of the active-set method per master problem.

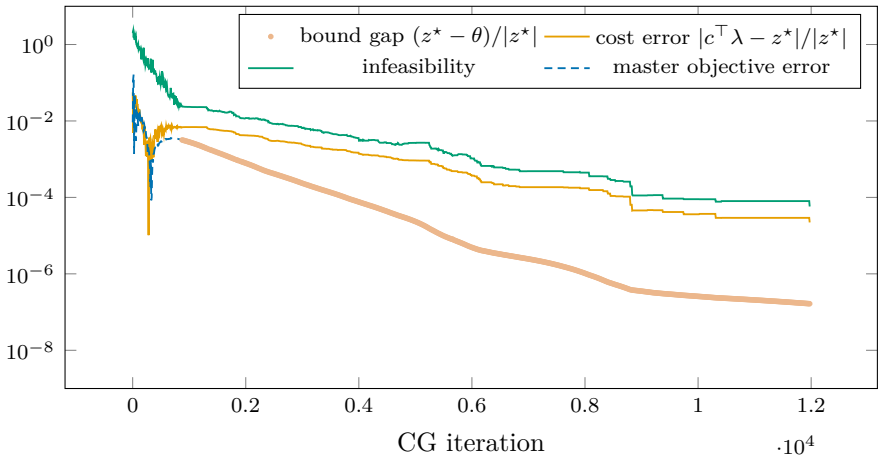


Figure 8.3: Optimality measures of the active-set method over a run (I2, equality variant). The Lagrangian bound is only available in iterations that price all pilots, which are the iterations that find no column.

8.5 Computational Time

Master time per iteration. Figure 8.4 shows the time of the master solve per column generation iteration on I2. The time of the LP master and of the two HiGHS QP masters grows with the number of columns. The HiGHS QP solvers take 100–170 ms per iteration in the second half of the run, since they rebuild their internal factorization for every master problem. The active-set method stays between 0.01 and 2 ms per iteration over the whole run, with a median of 0.08 ms in the second half. That is about three orders of magnitude below the HiGHS QP masters and more than two orders of magnitude below the warm-started LP master.

Where the time goes. Figure 8.5 splits the time of the active-set method into its operations. The largest share, 37–60%, is the direction computation (the right-hand side and the sparse solve with the factor). On the larger instances it is followed by the updates of the factorization and the periodic refactorization, together 35–50%. The multiplier test, the only operation whose cost grows with the number of columns, takes 10–17%. Pricing takes only 1–8%, except on the tiny instance I1.

Table 8.5 lists the events inside the active-set method for every run. Over a full run, the working set changes tens of thousands of times, and every change is a single update of the factorization. Representative swaps are rare, because representatives carry the largest weights.

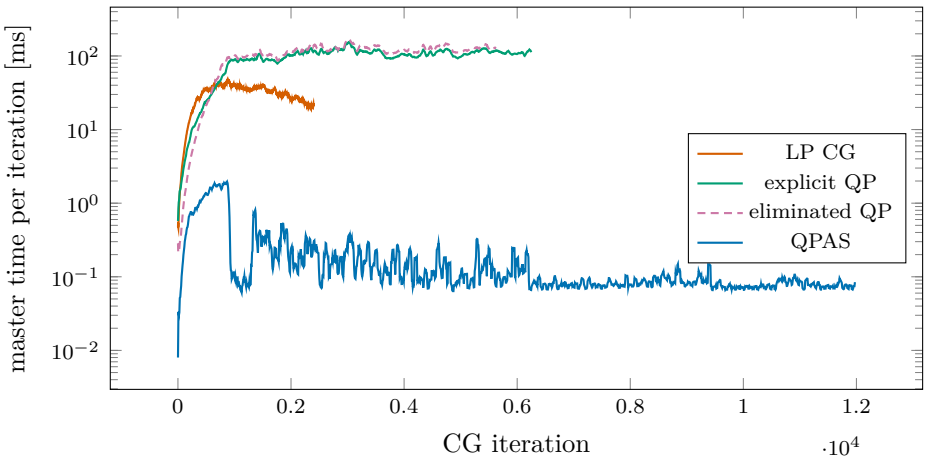


Figure 8.4: Master time per column generation iteration on I2 (moving average over 50 iterations).

Singular reduced Hessians occur, but only a handful of times per run, and each is resolved by one zero-curvature step. The final nullspace dimension $|\mathcal{N}_x|$ stays below the number of coverage rows, as guaranteed by Corollary 5.2.5.

		AS iter.	steps (full)	expans.	swaps	reduct.	surplus exp./red.	sing.	updates	refact.	$\lvert\mathcal{N}_x\rvert$	$\lvert\mathcal{N}_s\rvert$
I1	–	5 307	955 (564)	392	31	424	–	1	1 069	6	32	–
I1	30%	1 527	792 (455)	337	29	366	56 / 62	0	911	5	23	6
I2	–	62 209	34 807 (19 601)	15 209	215	15 428	–	3	60 401	221	219	–
I2	30%	33 709	21 599 (11 340)	10 260	173	10 480	1945 / 1979	1	35 405	149	186	34
I3	–	120 125	75 713 (40 619)	35 098	477	35 451	–	4	164 645	509	353	–
I3	30%	97 669	62 481 (33 115)	29 374	359	29 728	4049 / 4097	8	126 053	424	306	48
I4	–	210 375	138 784 (70 891)	67 898	667	68 375	–	5	378 479	966	477	–
I4	30%	194 685	128 451 (65 556)	62 899	466	63 378	11299 / 11361	4	316 764	878	417	62

Table 8.5: Events inside the active-set method over a whole column generation run: active-set iterations, primal steps (of which full steps), expansions, representative swaps, reductions, expansions/reductions of surpluses, singular steps, factorization updates, refactorizations, and the final sizes of $\mathcal{N}_x$ and $\mathcal{N}_s$.

Scaling. Figure 8.6 shows the total time of the active-set method and of the direct LP solve as the instances grow, up to an instance with 960 coverage rows (instances S5 and S6 with $|V| = 300$ and 400, $K = 16$ and 20, generated in the same way). Over the whole range, both grow roughly with the third power of m. For the active-set method this is expected: $|\mathcal{N}_x|$ grows proportionally to m, the factor of H_{eff} is dense (Section 8.7), and the number of iterations grows as well. From I4 on, the complete column generation run with the active-set method is faster than the direct solution of the full LP. On S6 the direct LP solve did not finish within 20 minutes, while the active-set method solved both variants in 4–7 minutes. The inequality variant is consistently faster than the equality variant, since it needs fewer iterations (Section 8.6).

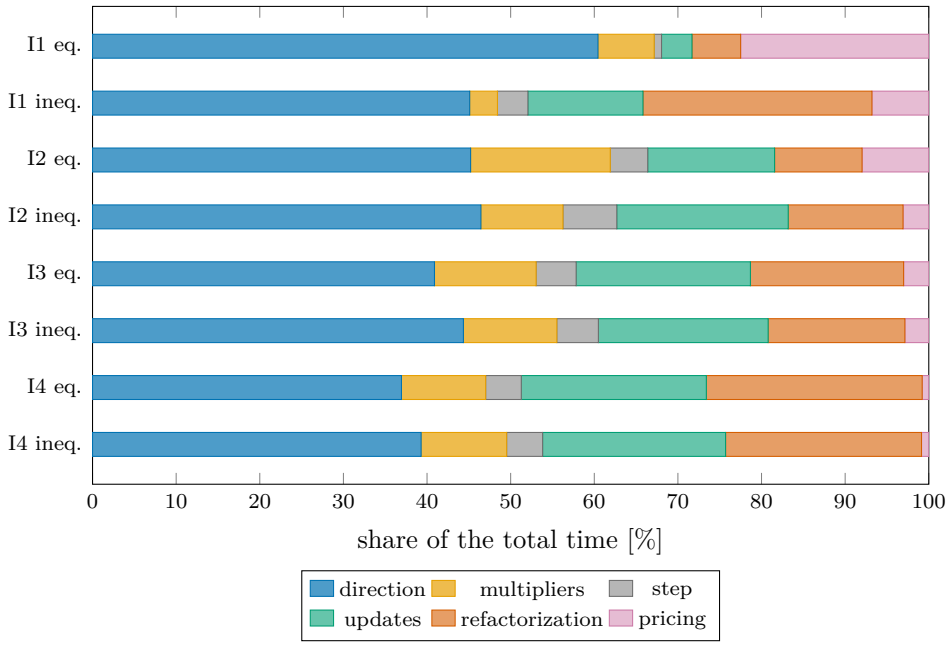


Figure 8.5: Time breakdown of the active-set method and of pricing, as shares of their sum. "updates" are the expansions and reductions of the working set without the refactorizations.

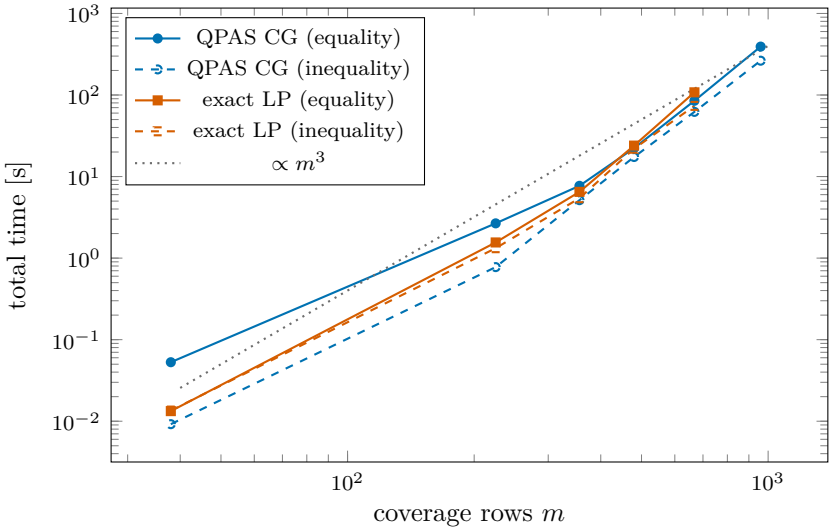


Figure 8.6: Total time against the number of coverage rows for instances I1–I4, S5 and S6. The exact LP of S6 did not finish within 20 minutes.

8.6 Inequality Rows

With 30% inequality rows, column generation needs considerably fewer iterations on all instances (Tables 8.2 and 8.3): over-covering a flight is allowed, so the optimal schedules are found with fewer columns, and the proximal phase is shorter.

Figure 8.7 follows the sizes of the nullspace sets over a run with inequality rows. Within the first iterations, about 35 surpluses are released: the initial schedules over-cover their rows, so the duals of these rows become negative. These rows drop out of A_{eff}, and by Corollary 5.2.5 they also reduce the room for nullspace columns, $|\mathcal{N}_x| \leq m - |\mathcal{N}_s|$. Over the rest of the run $|\mathcal{N}_s|$ only changes slowly, reaching 48, while individual surpluses are released and fixed again many times (Table 8.5). The number of nullspace columns grows during the column phase and ends at $|\mathcal{N}_x| = 306$, just below its bound $m - |\mathcal{N}_s| = 308$: at the end of the run, the effective reduced Hessian is almost as large as it can be.

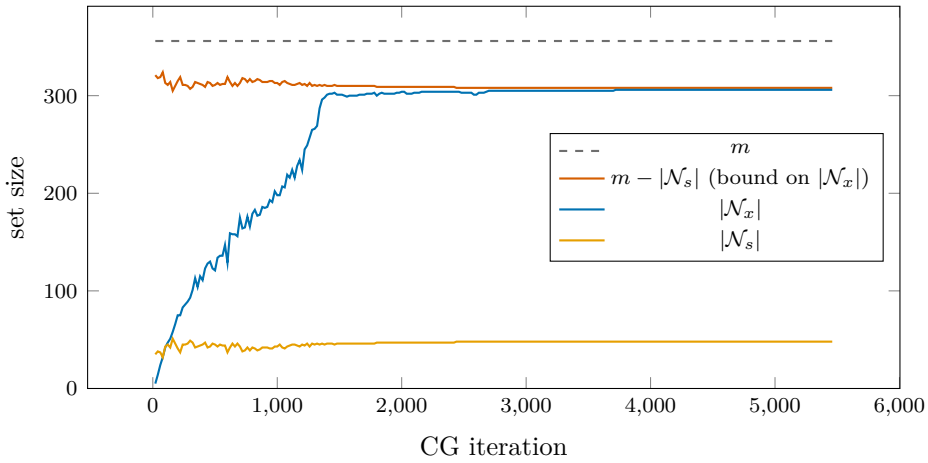


Figure 8.7: Nullspace columns and free surpluses over a run (I3, inequality variant), sampled every 20 iterations.

Surplus operations are frequent (Table 8.5): with inequality rows, 13–19% of all expansions and reductions of the working set on I2–I4 are expansions and reductions of surpluses, each of which is a rank-one update or downdate of the factorization. They do not make the method slower: the average time per master problem of the inequality variant is of the same order as that of the equality variant (Tables 8.2 and 8.3). The remaining differences come from the mix of iterations. On I2, for instance, the inequality variant has relatively few of the cheap center-only iterations (Table 8.4), which raises its average. Singular reduced Hessians, of columns or surpluses, occurred at most eight times per run. The singular surplus release of Theorem 7.3.2 is exercised by the test suite of the implementation on constructed instances.

8.7 Fill-In and Refactorization

The factorization of H_{eff} is updated tens of thousands of times per run (Table 8.5). Figure 8.8 shows what this does to its contents, using the quantities of Definition 6.5.1, without refactorization (left) and with refactorization every 150 updates (right).

The true factor is dense. In both panels the effective entries coincide with the dense equivalent: the factor of H_{eff} has essentially no exploitable sparsity. This confirms Remark 6.5.4: a fill-reducing ordering cannot help, and the cost of a solve grows quadratically with $|\mathcal{N}_x|$.

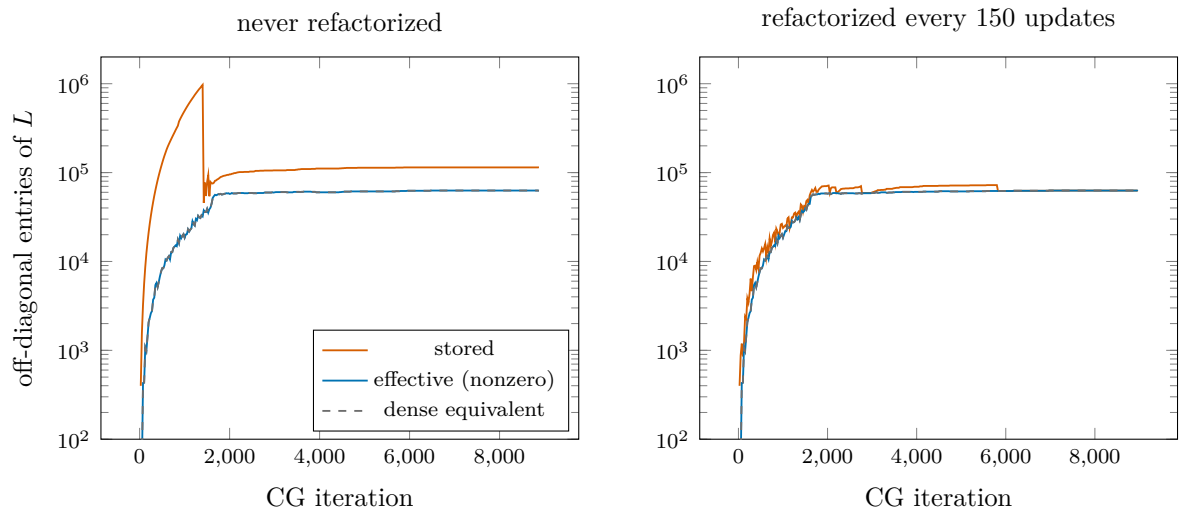


Figure 8.8: Contents of the factor L of H_{eff} over a run (I3, equality variant), sampled every 25 iterations: stored off-diagonal entries, the nonzero ones among them, and the off-diagonal entries of a dense factor of the same dimension.

Structural zeros accumulate. Without the periodic refactorization, the stored pattern only grows. On I3 it reaches almost a million entries in the column phase, more than fifteen times the size of the actual factor, and almost all of them are structural zeros. Every solve and every update pays for them, and they connect active indices with inactive ones, which enlarges the reach of the sparse solves. The drop around iteration 1 400 is a refactorization triggered by a degraded pivot (Section 7.2). It resets the pattern, after which it grows again and stays at about twice the size of the factor. With refactorization every 150 updates, the stored pattern stays within a small margin of the effective one throughout.

The refactorization interval. Table 8.6 and Fig. 8.9 vary the refactorization interval. Without refactorization, the method is three to five times slower and CHOLMOD needs six to eleven times more memory. Refactorizing too often is also expensive, since every rebuild costs $O(|\mathcal{N}_x|^3)$ (Proposition 6.5.3): with an interval of 25, I3 spends most of its time on almost 3 000 rebuilds. Between these extremes the optimum is flat. Any interval between 100 and 600 is within about 10% of the best time, and the default of 150 is within 4% of the best on both instances.

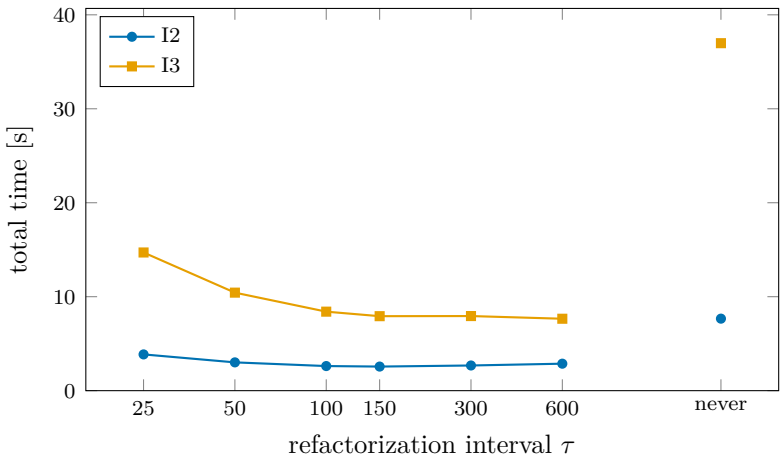


Figure 8.9: Total time of the active-set method against the refactorization interval.

	interval τ	time [s]	refact.	max. stored	max. struct. zeros	mean zero share	reach/$\lvert\mathcal{N}_x\rvert$	CHOLMOD peak [MB]
I2	never	7.66	2	361 143	343 768	62%	1.85	44.2
	25	3.86	1 299	25 626	2 420	4%	1.02	6.5
	50	3.01	662	27 945	4 179	7%	1.04	6.5
	100	2.62	326	31 572	8 288	17%	1.10	7.2
	150	2.56	221	33 872	10 226	15%	1.09	7.2
	300	2.68	110	40 987	17 636	32%	1.22	7.9
	600	2.87	56	48 151	25 544	42%	1.33	8.8
I3	never	36.98	5	971 176	939 055	52%	1.79	125.6
	25	14.71	2 989	63 500	3 465	2%	1.01	11.3
	50	10.43	1 502	67 105	5 965	6%	1.03	11.3
	100	8.41	764	71 179	9 840	8%	1.06	11.3
	150	7.93	509	72 309	13 305	10%	1.07	11.3
	300	7.94	261	82 932	21 358	23%	1.16	12.8
	600	7.66	130	94 279	34 994	21%	1.17	12.9

Table 8.6: Effect of the refactorization interval (equality variants). "reach/$\lvert\mathcal{N}_x\rvert$" is the average number of rows touched by a sparse solve per nullspace column. "never" still refactorizes when a pivot degrades.

8.8 Discussion

The experiments support the main claims of this work.

- **Correctness.** The active-set method solves the same sequence of master problems as a general-purpose QP solver, reproduces the optimum of the full linear program, and certifies it by the Lagrangian bound, for equality and inequality rows.
- **Speed.** A master problem of the active-set method costs between 0.01 and 7 ms on average on our instances. That is two to three orders of magnitude less than a HiGHS QP master and two orders of magnitude less than a warm-started LP master. This makes regularized column generation much faster than classical column generation, although it often needs more iterations: on I3, 7.7 s against 650 s.
- **Inequality rows** cost nothing extra: the surplus operations are rank-one updates of the same factorization, the factorized matrix only shrinks when surpluses become free, and the runs are shorter.
- **Refactorization** is essential for long runs, and a fixed interval of a few hundred updates works well.

The experiments also show the limits of the approach. The factor of H_{eff} is dense, so the cost of the method grows with the cube of the number of linking rows, and the **proximal phase** can be long, since the duals approach the restricted optimum in many small steps (Proposition 6.6.2). Two natural improvements are therefore a dense factorization with cheap rank-one updates once $\lvert\mathcal{N}_x\rvert$ is large, and an adaptive stabilization parameter in the proximal phase. A change of δ does not affect the factorization (Remark 5.2.8) and is therefore free for this method.

Chapter 9

Conclusion

This work developed a hot-started nullspace active-set method for the quadratic master problems of regularized column generation, for multi-block linear programs whose linking constraints may be equalities or inequalities of either direction.

9.1 Summary

The starting point was a **unified formulation** (Chapter 2): surplus variables bring every linking row into equality form, and the sign conditions of the inequality rows become ordinary dual constraints of the surplus columns. Proximal regularization of the dual then leads to a convex quadratic master problem (Chapter 3). Because all linking rows are equalities, its auxiliary variable can be eliminated for all rows, and the result has only the convexity constraints and nonnegativity bounds. Two identities underlie everything that follows: the duals of the linking rows are an explicit affine function of the primal iterate, and the gradient of the quadratic objective consists of the reduced costs of the columns and the duals of the inequality rows.

For the active-set method (Chapter 4), the constraint matrix of a working set admits a **reordering into five sets**: representatives, working-set columns, nullspace columns, active surpluses and free surpluses. The surpluses violate the column generation property of the convexity matrix, but this only adds two groups of columns with identity structure. The reordering yields an explicit nullspace basis of vectors $e_j - e_{\mathrm{rep}(j)}$ and unit vectors, which is never stored (Chapter 5). The reduced KKT system **decouples**: the surplus part of the direction is explicit, and only the **effective reduced Hessian** $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$ has to be factorized. Here A_{eff} holds the equality rows and the inequality rows with active surplus. The column part of the method is thus the method for equality rows, applied to a linking matrix whose rows change with the surplus working set. The Lagrange multipliers are the reduced costs and the dual signs, so the optimality test of the master problem and the pricing test are the same test.

Every change of the working set is a row addition, a row deletion or a rank-one modification of a fixed-size $LDL^\top$ factorization (Chapter 6). A new column or a new stability center leaves the factorization unchanged, which gives a hot start of every master problem. Structural zeros left by the updates are controlled by periodic refactorization in increasing index order. Singular reduced Hessians are resolved by an exact zero-curvature step, which removes the dependency and keeps every working-set variable at its bound, for released columns and released surpluses alike (Chapter 7).

On generated crew scheduling instances (Chapter 8), the method reproduces the optimum of the full linear program and certifies it by the Lagrangian bound. Its master problems cost between

0.01 and 7 ms on average. That is two to three orders of magnitude less than a general-purpose QP solver, and fast enough to make regularized column generation much faster than classical column generation with a warm-started LP master: 7.7 s instead of 650 s on the largest instance on which both were run. From medium-sized instances on, the complete column generation run is even faster than solving the full linear program directly. Inequality rows are handled at no extra cost and lead to shorter runs.

9.2 Outlook

The experiments point to three directions for further work.

- **Dense factorization for large nullspaces.** The factor of H_{eff} turned out to be dense. For large $|\mathcal{N}_x|$, a dense $LDL^\top$ (or QR) factorization with rank-one updates would avoid the overhead of sparse data structures and would make the periodic refactorization unnecessary.
- **Adaptive stabilization in the proximal phase.** The long tail of center-only iterations is a property of the proximal point method with a fixed δ (Proposition 6.6.2). Since δ does not enter the factorization, it can be decreased in this phase at no cost to the linear algebra.
- **Range rows and inexact master solves.** Linking rows with two finite bounds need surpluses with upper bounds, which fit into the same framework as an additional working-set state. Moreover, since the multipliers are the reduced costs, pricing could be called at any subspace minimizer instead of only at the optimum of the master problem. This would turn column generation into a single active-set method over all columns.

Bibliography

[1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[2] O. Briant, C. Lemaréchal, Ph. Meurdesoif, S. Michel, N. Perrot, and F. Vanderbeck. Comparison of bundle and classical column generation. *Mathematical Programming*, 113(2):299–344, 2008.

[3] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software*, 35(3):22:1–22:14, 2008.

[4] G. B. Dantzig and P. Wolfe. Decomposition principle for linear programs. *Operations Research*, 8(1):101–111, 1960.

[5] T. A. Davis and W. W. Hager. Modifying a sparse Cholesky factorization. *SIAM Journal on Matrix Analysis and Applications*, 20(3):606–627, 1999.

[6] T. A. Davis and W. W. Hager. Row modifications of a sparse Cholesky factorization. *SIAM Journal on Matrix Analysis and Applications*, 26(3):621–639, 2005.

[7] G. Desaulniers, J. Desrosiers, and M. M. Solomon, editors. *Column Generation*. Springer, New York, 2005.

[8] O. du Merle, D. Villeneuve, J. Desrosiers, and P. Hansen. Stabilized column generation. *Discrete Mathematics*, 194(1–3):229–237, 1999.

[9] R. Fletcher. A general quadratic programming algorithm. *IMA Journal of Applied Mathematics*, 7(1):76–91, 1971.

[10] P. E. Gill, G. H. Golub, W. Murray, and M. A. Saunders. Methods for modifying matrix factorizations. *Mathematics of Computation*, 28(126):505–535, 1974.

[11] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[12] J. E. Kelley. The cutting-plane method for solving convex programs. *Journal of the Society for Industrial and Applied Mathematics*, 8(4):703–712, 1960.

[13] K. C. Kiwiel. Proximity control in bundle methods for convex nondifferentiable minimization. *Mathematical Programming*, 46(1–3):105–122, 1990.

[14] M. E. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.

[15] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

[16] R. T. Rockafellar. Monotone operators and the proximal point algorithm. *SIAM Journal on Control and Optimization*, 14(5):877–898, 1976.